

Laboratorio di Programmazione I

Con il linguaggio Python



ARTIFICIAL INTELLIGENCE
& DATA ANALYTICS



UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

Laura Nenzi, Giulio Caravagna, Stefano Alberto Russo

Corso di Laurea Triennale in Intelligenza Artificiale e Data Analytics (AIDA)

Dipartimento di Matematica, Informatica e Geoscienze

Università di Trieste

Dispense trascritte dalle lezioni grazie ai contributi cumulativi di: Gianluca Guglielmo (stesura iniziale), Michele Rispoli, Pietro Morichetti, Nicolas Solomita, Andrea Mecchina, Elena Buscaroli, Federico Pigozzi, Lucrezia Valeriani e Valentina Blasone. Materiale distribuito sotto licenza Creative Commons Attribution-ShareAlike 4.0 International License¹

Versione 1.0.1 (16 marzo 2026)



¹La licenza dice grossomodo che, a condizione di citare l'autore e di utilizzare la stessa tipologia di licenza, questi materiali possono essere liberamente copiati, modificati, usati per creare opere derivate e ridistribuiti.

Indice

I	Strumenti di lavoro	1
II	Introduzione a Python	4
1	Tipi di dato ed operatori	5
1.1	Operatori	6
1.2	Stringhe	7
1.3	Liste	8
1.4	Dizionari	9
1.5	Tuple	10
1.6	Insiemi (Set)	11
2	Indentazione, istruzioni condizionali e cicli	11
2.1	Indentazione	11
2.2	Istruzioni condizionali	11
2.3	Cicli	12
3	Funzioni	13
3.1	Scope	14
3.2	Moduli	15
4	Essere pythonici	16
5	File e database	17
III	Programmazione ad oggetti	23
6	Programmazione ad oggetti	23
6.1	Classi ed oggetti	23
6.2	Ereditarietà tra classi	30
7	Meccanismi avanzati di iterazione	41
7.1	List comprehensions	41
7.2	Iteratori	44

7.3	Liste concatenate in Python	50
IV	Gestione degli errori e dell'input	53
8	Le eccezioni	53
9	Controllo degli input	58
V	Esercizi	63
10	Ereditarietà	63
11	List comprehension	67
12	Iteratori	67

Preambolo. Questa dispensa costituisce il materiale didattico per il corso di *Laboratorio di Programmazione Modulo 1*, relativo all'anno accademico di riferimento. Il corso è un modulo del corso *Introduzione alla Programmazione e Laboratorio*.

A seconda delle annualità e del corso di laurea, il corso prevede 9, 12 o 15 CFU e si articola in due o tre moduli: uno dedicato alla programmazione (oggetto della dispensa del prof. Caravagna) e uno (o due) al laboratorio (oggetto della presente dispensa).

Questo modulo di laboratorio si concentra interamente su **Python**. Il primo modulo è propedeutico a questo quindi viene assunto che lo studente sia già familiare con i costrutti e gli operatori di base di un linguaggio di programmazione. Verranno inoltre affrontati i fondamenti della programmazione a oggetti e alcuni concetti di programmazione funzionale, entrambi trattati in **Python**.

Gli obiettivi formativi e i materiali didattici possono variare di anno in anno: non sempre l'intera dispensa sarà utilizzata, e talvolta potranno essere forniti contenuti integrativi. In linea generale, il modulo di programmazione mira a introdurre i concetti essenziali del *computational thinking*, con particolare attenzione ai principi di ragionamento e progettazione che restano validi indipendentemente dal linguaggio utilizzato, mentre il laboratorio approfondisce l'uso di **Python**, affrontando tematiche complementari e fornendo competenze pratiche che si affiancano alla solida base concettuale costruita nel modulo di programmazione, utile per affrontare con agilità diversi linguaggi.

L'intelligenza artificiale (IA) scrive codice, ma non capisce il vostro problema. L'IA produce testo plausibile anche se non sapete formalizzare il problema, ed il codice plausibile è pericoloso: compila, ma fa la cosa sbagliata. Senza esercizio non sviluppate il “debug mentale”, cioè la capacità di leggere un algoritmo e chiedervi: ha senso? copre i casi limite? cosa succede se l'input è sporco? Programmare non è scrivere righe, è pensare in modo strutturato. Bisogna decidere cosa fare prima di decidere come farlo. Gli esercizi allenano la decomposizione del problema, potete vederla come una ginnastica cognitiva. La verità è che oggi programmare è ancora più importante, non meno. Perché ora dovete saper valutare, correggere, integrare codice generato automaticamente. È un ruolo più maturo: da scrittori a ingegneri del ragionamento. Se vogliamo dirla in modo quasi filosofico: l'IA è un acceleratore di pensiero. Ma se non c'è pensiero, accelera il vuoto. E il vuoto, quando lo accelera, resta vuoto. Per cui vi consiglio vivamente di provare a fare gli esercizi usando l'IA in maniera limitata.

Libri di testo. Durante la prima lezione e nella pagina del corso sono forniti dei riferimenti testuali aggiornati ogni anno. Gli studenti che desiderino consultare testi di riferimento più strutturati possono rivolgersi al docente per ricevere indicazioni sui titoli più appropriati.

Guida alla dispensa. Questa dispensa si “evolve” di anno in anno, e pertanto a volte potrebbe contenere errori. Dubbi, chiarimenti ed eventuali errori possono essere segnalati via mail a lnenzi@units.it. È prevista una forma di ricompensa per la segnalazione di errori (non banali).

Nel testo le spiegazioni sono colloquiali, sacrificando il rigore per la chiarezza espositiva per fornire un accesso semplice e diretto ai concetti base. Esempi di codice sono colorati e discussi. In generale una variabile indicata come x , y , etc. denota una quantità matematica, mentre x o y una variabile scritta nel programma, quindi codice. Spesso le due coincidono e la notazione non è necessariamente consistente tra le varie sezioni.

Strumenti di lavoro

PART

I

Per affrontare il *Laboratorio di Programmazione in Python* è necessario usare alcuni strumenti, che vediamo brevemente.

File manager Il *File Manager* è un software che consente di gestire e organizzare i file e le cartelle presenti in un sistema operativo. In pratica, è quello strumento che permette all'utente di vedere, modificare, copiare ed eliminare file e cartelle. Esempi sono Windows Explorer (Esplora file) su Windows, il Finder su MacOS e Nautilus o Dolphin sui sistemi Linux.

Shell La *shell* permette di eseguire programmi e navigare il file system attraverso una linea di comando, *Command-Line Interface (CLI)*, ovvero di interagire direttamente con la macchina. Nei sistemi Windows le shell a disposizione sono il *Prompt dei Comandi (CMD)* e la *PowerShell*, mentre nei sistemi Unix si ha *BASH* (in genere accessibile tramite l'applicazione *Terminale*).

Editor del codice L'*Editor del Codice* può essere un qualsiasi programma per la scrittura del testo *semplice*, ovvero che non aggiunga della formattazione come il grassetto, corsivo, paragrafi etc. È possibile usare un editor che interpreti il linguaggio utilizzato per colorare le diverse sintassi e facilitarne la lettura. Per il corso è necessario impostare l'editor in modo che utilizzi 4 spazi al posto dei tab, che serviranno per indentare il codice. Python segue la convenzione PEP 8 (le linee guida ufficiali di stile) che raccomanda 4 spazi per livello di indentazione, un tab può essere interpretato in modo diverso da sistemi diversi.

IDE L'*Integrated Development Environment* è un editor che supporta lo sviluppatore attraverso l'integrazione con sistemi di debugging, File Manager, Shell, talvolta Git e svariate altre agevolazioni, le quali rendono la programmazione più scorrevole. Gli IDE più diffusi supportano diversi linguaggi e possono adattare i propri suggerimenti a seconda della sintassi utilizzata. Tra i più utilizzati troviamo Sublime e Code::Blocks.

Visual Studio Code *Visual Studio Code* (VS Code) è un editor di codice estensibile sviluppato da Microsoft. Non è propriamente un IDE tradizionale, bensì un editor leggero che può essere arricchito tramite estensioni per supportare numerosi linguaggi di programmazione. Integra un terminale,

strumenti di debugging e il supporto al versionamento tramite *Git*. La sua architettura modulare consente di aggiungere funzionalità come completamento automatico, analisi statica del codice e gestione di ambienti virtuali. Grazie al sistema di estensioni e al *Language Server Protocol*, VS Code può assumere il comportamento di un vero e proprio ambiente di sviluppo integrato.

Git Il software per il versionamento *Git* è uno strumento per tenere traccia delle modifiche fatte al codice, può essere sincronizzato con dei collaboratori ed è decentralizzato. Le *Repository* (repo) sono le cartelle dei progetti e possono contenere sia codice che file di altro tipo. L'operazione base è il *commit*, che crea un “salvataggio” del codice in Git permettendo di avere dei checkpoint, univocamente identificati da un cosiddetto *hash*. È possibile trovare [qui](#) una guida su Git scritta da Michele Rispoli.

GitHub *GitHub* è una piattaforma online che ospita repository *Git* e facilita la collaborazione tra sviluppatori. Permette di condividere progetti, gestire modifiche tramite *pull request*, discutere problemi attraverso le *issue* e integrare sistemi di automazione per test e distribuzione del software. Oltre all'hosting del codice, offre strumenti per la documentazione, la revisione collaborativa e la gestione di progetti.

GitHub Codespaces *GitHub Codespaces* è un servizio cloud che consente di creare ambienti di sviluppo remoti direttamente collegati a una repository GitHub. L'ambiente viene eseguito su una macchina virtuale nel cloud e può essere utilizzato tramite browser oppure collegandosi con *Visual Studio Code*. In questo modo lo sviluppo non dipende dalle risorse locali del computer, ma da un ambiente configurato e riproducibile, accessibile da qualsiasi dispositivo.

Ambiente virtuale Un *ambiente virtuale* è uno spazio isolato che contiene una specifica versione di Python e le librerie necessarie a un determinato progetto. Ogni progetto può avere il proprio ambiente, indipendente dagli altri e dal sistema operativo. Questo isolamento è fondamentale per evitare conflitti tra versioni diverse delle stesse librerie o tra progetti che richiedono configurazioni differenti.

Nel corso utilizzeremo *Miniconda* per creare e gestire ambienti virtuali. In particolare, verrà creato un ambiente dedicato al laboratorio, all'interno del quale verranno installate solo le librerie necessarie. Quando un ambiente è attivo, il terminale mostra il suo nome tra parentesi all'inizio della riga di comando: ciò indica che i comandi Python e le installazioni di librerie verranno eseguiti all'interno di quell'ambiente.

È buona pratica non lavorare mai nell'ambiente **base**, ma creare un ambiente specifico per ciascun progetto.

Pseudocodice Un ultimo strumento concettuale è costituito dal cosiddetto *pseudocodice*. Prima di scrivere il codice vero e proprio, in qualsiasi linguaggio di programmazione, è utile scriverne una bozza, anche a carta e penna, focalizzandosi non sul come, ma sul *cosa* fare. Non c'è uno standard con cui scrivere questa bozza: lo pseudocodice è del tutto inventato, usando parole anche in linguaggio naturale (italiane o inglesi) che permettano di trasporre il problema che si cerca di risolvere in logica di programmazione imperativa ma senza, appunto, preoccuparsi di dettagli come la sintassi, i tipi dati etc. per non interrompere il flusso di ragionamento logico.

Introduzione a Python

PART

II

Cos'è Python *Python* è un linguaggio di programmazione interpretato di alto livello, cioè "distante" dal linguaggio macchina. Durante il corso useremo la versione 3, e in particolare ≥ 3.6 . Python nasce nel 1991 come linguaggio di programmazione ad oggetti. Il suo utilizzo è in costante crescita, grazie anche alla sua semplicità e intuitività. È un linguaggio *interpretato*, quindi non ha bisogno di compilazione e può essere anche usato in modalità interattiva dall'interprete Python, molto utile ogniqualvolta si voglia testare delle cose in rapidità.

Python è anche il linguaggio “de facto” standard per la Data Science, grazie ad un enorme ecosistema di strumenti per il calcolo numerico, scientifico e statistico.

In questo corso si assume che si abbia già familiarità con i costrutti e gli operatori base di un linguaggio di programmazione, come ad esempio:

- Assegnazione di variabili e stampa a schermo (`=`, `print`)
- Operatori condizionali (`if`, `else`)
- Operatori aritmetici (`+`, `-`, `*`, etc.)
- Operatori logici (`and`, `or`)
- Operatori di confronto (`==`, `<`, `>`, etc.)
- Cicli (`for`, `while`, etc.)

Nei prossimi capitoli verranno comunque presentati questi costrutti e operatori nel linguaggio Python.

Ciao, mondo! Come ogni buona trattazione di un linguaggio di programmazione, iniziamo con un classico “Ciao mondo”.

Funzione 1 (Print). La funzione `print` permette di stampare una stringa a schermo. La sintassi è la seguente:

```
1 print('Ciao, Mondo!') # Stampa "Ciao, Mondo!"
```

È anche possibile inserire un valore personalizzato nella stringa usando la sintassi `.format`:

```
1 x = 25
2 print('Ho {} anni.'.format(x)) # Stampa 'Ho 25 anni.'
```

1 Tipi di dato ed operatori

In Python non è necessario definire il tipo di dato delle variabili durante la dichiarazione. Infatti, Python deduce automaticamente il tipo durante l'inizializzazione grazie al valore che le viene assegnato. E' infatti un linguaggio *non tipizzato*. Al contrario dei linguaggi *tipizzati* come ad esempio il C, in Python una variabile può cambiare tipo dati in continuazione, basta assegnarle un nuovo valore di tipo diverso.

In Python sono presenti i seguenti tipi dati principali (ma ce ne sono anche altri più complessi, come liste e dizionari, che vedremo in seguito):

```
1 mia_var = 1      # variabile di tipo intero (int)
2 mia_var = 1.1    # variabile di tipo floating point (float)
3 mia_var = 'ciao' # variabile di tipo stringa (str)
4 mia_var = True   # variabile di tipo booleano (bool)
5 my_var = [1, 2, 3]      # variabile di tipo lista (list)
6 my_var = (1, 2, 3)      # variabile di tipo tupla (tuple)
7 my_var = {1, 2, 3}      # variabile di tipo insieme (set)
8 my_var = {"a": 1, "b": 2} # variabile di tipo dizionario (dict)
9 mia_var = None        # "niente" si rappresenta con il None
```

Nel codice sopra, i cancelletti indicano un commento: ogni carattere successivo ad un cancelletto viene ignorato dall'interprete Python, fino a nuova riga.

In Python il tipo dati `None` viene usato, oltre al valore nullo, anche per semplicemente indicare qualcosa di non rilevante, come ad esempio una variabile che è necessario inizializzare per motivi sintattici ma che non è ancora stata utilizzata (p.es. `somma_totale=None`).

Il tipo dato di una variabile può essere controllato usando la funzione `type`. La chiamata `type(var)` ritorna il tipo della variabile.

Esempio 1. Esempi di utilizzo della funzione `type`:

```
1 x = 10
2 print(type(x))      # <class 'int'>
3
4 y = 3.14
5 print(type(y))      # <class 'float'>
6
7 z = 'ciao'
8 print(type(z))      # <class 'str'>
```

Spesso è necessario convertire un valore da un tipo ad un altro. Questa operazione prende il nome di *casting* e si effettua richiamando il tipo di destinazione

come fosse una funzione.

Esempio 2. Esempi di casting tra tipi diversi:

```
1 a = '25'
2 b = int(a)          # da stringa a intero
3
4 c = 3.7
5 d = int(c)          # da float a intero (troncamento)
6
7 e = 10
8 f = float(e)        # da intero a float
9
10 g = str(e)         # da intero a stringa
```

Se il valore non è compatibile con il tipo di destinazione, viene generato un errore.

1.1 Operatori

Python dispone di diversi operatori built-in (cioè predefiniti e disponibili senza dover importare niente) per effettuare operazioni su valori e variabili. per effettuare operazioni su valori e variabili. Di seguito vediamo gli operatori aritmetici, di confronto e logici ma ce ne sono anche altri che vedremo in seguito.

Sintassi 1 (Operatori aritmetici). Gli operatori aritmetici effettuano le semplici operazioni matematiche indicate dal simbolo.

Operatore	Nome	Esempio
+	Addizione	x+y
-	Sottrazione	x-y
*	Moltiplicazione	x*y
/	Divisione	x/y
%	Modulo ^a	x%y
**	Esponenziale	x**y

^aL'operazione modulo ritorna il resto della divisione x/y .

Sintassi 2 (Operatori di confronto). Gli operatori di confronto ritornano True o False a seconda che la condizione indicata sia rispettata o no. Sono usati soprattutto nei costrutti if.

Operatore	Nome	Esempio
==	Uguale	x==y
!=	Diverso	x!=y
>	Maggiore	x>y
<	Minore	x<y
>=	Maggiore o uguale	x>=y
<=	Minore o uguale	x<=y

Sintassi 3 (Operatori logici). Gli operatori logici permettono di collegare due (o più) condizioni attraverso le logiche `and`, `or` e `not`.

Operatore	Descrizione	Esempio
<code>and</code>	True se entrambe True	<code>x<5 and x<10</code>
<code>or</code>	True se almeno una True	<code>x<5 or x<4</code>
<code>not</code>	Inverte il risultato	<code>not(x<5 and x<10)</code>

1.2 Stringhe

Una *stringa* (`str`) è una sequenza *immutabile* di caratteri. In Python le stringhe si possono delimitare sia con apici singoli `'...'` sia con doppi apici `"..."` (scegline uno stile e usalo in modo coerente). Essendo iterabili, supportano gli operatori di appartenenza, l'accesso per indice e lo *slicing*, esattamente come le liste che vedrete subito dopo.

Esempio 3. Alcune stringhe in Python:

```

1 s1 = 'Ciao'
2 s2 = "Mondo"
3 s3 = 'Python 3.12'
4
5 print(s1 + ' ' + s2)    # concatenazione: 'Ciao Mondo'
6 print(len(s1))         # lunghezza: 4

```

Sintassi 4 (Accesso e slicing su stringhe). Come per le liste, posso accedere ai caratteri per posizione e fare slicing:

```

1 s = 'Programmazione'
2
3 s[0]    # 'P' (primo carattere)
4 s[-1]   # 'e' (ultimo carattere)
5 s[0:7]  # 'Program'
6 s[7:]   # 'mazione'

```

Sintassi 5 (Immutabilità). Le stringhe sono immutabili: non posso modificare un carattere "in place".

```
1 s = 'ciao'
2 # s[0] = 'C'      # ERRORE: le stringhe sono immutabili
3 s = 'C' + s[1:]   # OK: creo una nuova stringa
```

Sintassi 6 (Formattazione di stringhe). Per inserire valori nelle stringhe si può usare `.format()` oppure le *f-string* (consigliate per leggibilità):

```
1 x = 25
2 print('Ho {} anni.'.format(x))
3 print(f'Ho {x} anni.')
```

Input dell'utente In Python esiste una funzione built-in chiamata `input` che sospende l'esecuzione del programma e attende che l'utente inserisca un valore da tastiera. Il valore inserito viene sempre ritornato come *stringa* (`str`), quindi se serve un numero è necessario effettuare un casting.

Funzione 2 (`Input`). La funzione `input` legge una riga da tastiera. La sintassi base è la seguente:

```
1 testo = input()      # attende l'input dell'utente
2 print(testo)         # stampa ciò che l'utente ha scritto
```

Prima dell'inserimento dei dati è buona norma visualizzare un messaggio (*prompt*) che informi l'utente di cosa deve inserire:

```
1 nome = input('Come ti chiami?\n')
2 print('Ciao, {}'.format(nome))
```

Poiché `input` ritorna sempre una stringa, per ottenere valori numerici bisogna convertire il risultato:

```
1 eta = int(input('Quanti anni hai?\n')) # casting da str a int
2 print('Tra un anno avrai {} anni'.format(eta + 1))
```

1.3 Liste

La *lista* è un tipo dati built-in che permette di rappresentare una sequenza mutabile di oggetti. Gli oggetti possono essere di tipo eterogeneo, anche se questo tipo di lista non serve in quasi nessun caso. Una lista si definisce usando le parentesi quadre.

Esempio 4. Alcune liste in Python:

```
1 mia_lista = [1,2,3] # Lista di numeri
2 mia_lista = ['Marco', 'Irene', 'Paolo'] # Lista di stringhe
```

```
3 mia_lista = [15, 32, 'ambo'] # Lista eterogenea
```

Le liste ci introducono agli operatori di appartenenza, all'accesso per posizione e allo *slicing*.

Sintassi 7 (Operatori di appartenenza). Gli operatori di appartenenza servono a controllare se un valore si trova in una lista, stringa o tupla^a.

Operatore	Descrizione	Esempio
in	True se x è nella lista y	x in y
not in	Inverte risultato di in	x not in y

^aUna tupla è una lista *immutabile* definita usando le parentesi tonde.

Sintassi 8 (Accesso per posizione). Posso accedere agli elementi di una lista per posizione usando la notazione con parentesi quadre:

```
1 mia_lista[0] # Primo elemnto
2 mia_lista[1] # Secondo elemento
3 mia_lista[-1] # Ultimo elemento
```

Con la stessa sintassi si può accedere agli elementi delle stringhe, che si possono immaginare come una serie (lista) di caratteri.

Sintassi 9 (Slicing). È possibile effettuare lo *slicing* delle liste, ovvero tagliarne una fetta, usando la seguente sintassi^a:

```
1 mia_stringa[0:50] # Dal 1° al 50° carattere
2 mia_stringa[30:50] # Dal 30° al 50° carattere
3 mia_stringa[0:-1] # Dal 1° al penultimo carattere
```

Si può applicare lo slicing anche alle stringhe, che sono immaginabili come una serie di caratteri.

^aIn questa sintassi, l'estremo sinistro dell'intervallo è compreso, l'estremo destro è escluso.

1.4 Dizionari

Il *dizionario* è un tipo dati built-in che permette di associare un valore **value** ad una chiave **key**. Si definisce usando le parentesi graffe e separando i **value** e le **key** con i due punti. Si accede al valore associato ad una chiave utilizzando le parentesi quadre.

Esempio 5. Nel seguente esempio, alcuni dizionari in Python:

```
1 mio_diz = {'Trieste': 34100, 'Padova': 35100} # Diz. di numeri
2 mio_diz = {'Trieste': 'TS', 'Padova': 'PD'} # Diz. di stringhe
```

```

3
4 sigla_ts = mio_diz['Trieste'] # A sigla_ts sarà assegnato il
5                               # valore legato alla chiave
6                               # 'Trieste' nel dizionario
                               ↪ my_dict

```

Le chiavi devono essere tipi dati che possono essere univocamente identificati (“hashabili”). Non approfondiamo ora questo concetto, come prima approssimazione diciamo che possiamo usare come chiave tutti i tipi dati “base” non composti, ovvero interi, stringhe, floating point ma non le liste. I valori, invece, possono essere di un tipo dati qualsiasi, incluso liste e dizionari: possiamo infatti tranquillamente creare un dizionario di liste o un dizionario di altri dizionari, a patto che le chiavi siano impostate come descritto sopra. Come per le liste è possibile usare gli operatori di appartenenza per controllare se una chiave è contenuta nel dizionario. Non è possibile utilizzare gli operatori di appartenenza per i valori.

1.5 Tuple

La *tupla* è un tipo dati built-in che rappresenta una sequenza *immutabile* di elementi. Si definisce usando le parentesi tonde. A differenza delle liste, una volta creata non può essere modificata.

Esempio 6. Alcune tuple in Python:

```

1 mia_tupla = (1, 2, 3)
2 mia_tupla = ('Marco', 'Irene', 'Paolo')
3 mia_tupla = (10, 'ciao', 3.14)

```

Come per le liste, è possibile accedere agli elementi per posizione e usare lo slicing:

```

1 mia_tupla[0]    # Primo elemento
2 mia_tupla[-1]   # Ultimo elemento
3 mia_tupla[0:2]  # Slice

```

Le tuple vengono spesso utilizzate per rappresentare dati che non devono essere modificati, oppure per restituire più valori da una funzione.

```

1 def esempio():
2     return (1, 2)
3
4 a, b = esempio() # unpacking della tupla

```

1.6 Insiemi (Set)

Il *set* è un tipo dati built-in che rappresenta un insieme di elementi *non ordinati* e *senza duplicati*. Si definisce usando le parentesi graffe oppure la funzione `set()`.

Esempio 7. Esempi di insiemi:

```
1 mio_set = {1, 2, 3}
2 mio_set = {1, 2, 2, 3} # diventa {1, 2, 3}
```

I set sono utili per verificare appartenenza e per effettuare operazioni insiemistiche:

```
1 A = {1, 2, 3}
2 B = {3, 4, 5}
3
4 A | B    # unione
5 A & B    # intersezione
6 A - B    # differenza
```

Non è possibile accedere agli elementi di un set per indice, poiché non sono ordinati.

2 Indentazione, istruzioni condizionali e cicli

2.1 Indentazione

Nella sintassi di C e C++ un blocco di codice contenente le istruzioni di un costrutto `if` o di un ciclo è delimitato da due parentesi graffe. In Python invece, l'inizio e la fine di un blocco di codice è definito dall'*indentazione* del blocco stesso.

Indentare significa “far rientrare” alcune parti del listato di codice, in genere tramite degli spazi. È una pratica comune in tutti i linguaggi di programmazione, ma in Python è strettamente legata al funzionamento del linguaggio stesso ed è da considerarsi parte della sintassi. Di norma, vengono usati 4 spazi o un carattere `tab` (l'importante è essere coerenti in tutto il codice), ma gli spazi sono da preferirsi. Vanno indentanti anche i commenti allo stesso livello del blocco di codice cui si riferiscono, per semplificare la lettura.

2.2 Istruzioni condizionali

Un'*istruzione condizionale* (il classico `if-else`) permette di inserire una condizione, che deve essere verificata per poter eseguire le istruzioni contenute nelle successive righe di codice.

Nota bene: la riga contenente l'istruzione condizionale termina con due punti, che la separano dal blocco di codice che contiene le istruzioni da eseguire nel caso in cui la condizione sia verificata e che deve essere correttamente indentato.

Esempio 8. Un semplice costrutto if:

```
1 if (mia_var > tua_var):
2     # Indentazione per segnalare che queste
3     # istruzioni fanno parte del blocco if
4     print("La mia var è più grande della tua")
5     if (mia_var-tua_var) <= 1:
6         # Ulteriore indentazione per il secondo if
7         print("...Non così tanto...")
```

Note bene: in Python è stata introdotta la parola chiave `elif` per comprimere la scrittura di `else if`.

Esempio 9. Un costrutto if-elif-else:

```
1 if (mia_var > tua_var):
2     print("La mia var è più grande della tua")
3     if (mia_var-tua_var) <= 1:
4         print("...Non così tanto...")
5     elif (mia_var-tua_var) <= 5:
6         print("...Abbastanza")
7     else:
8         # L'ultima condizione ha bisogno solo dell'else, che
9         # gestisce tutti i casi che non sono stati considerati
10        print("...Di molto")
```

2.3 Cicli

Anche in Python abbiamo i classici cicli `for` e `while`. Anche in questo caso la prima riga termina con i due punti ed è seguita da un blocco indentato, contenente le istruzioni da eseguire per ogni iterazione del ciclo.

Esempio 10. Un ciclo for:

```
1 for i in range(10):
2     # Indentazione per identificare il blocco del for
3     print(i)
```

Esempio 11. Un ciclo while:

```
1 i = 0
2 while i<10:
3     # Indentazione per identificare il blocco del while
```

```
4 print(i)
5 i = i + 1
```

Funzione 3 (Range). La funzione built-in `range`, introdotta nel `for` dell'esempio precedente, crea “al volo” una lista di numeri su cui poi si può effettuare un ciclo, nel seguente modo^a:

```
1 range(10)      # Lista di numeri da 0 a 9
2 range(3, 10)   # Lista di numeri da 3 a 9
3 range(3, 10, 2) # Lista di numeri da 3 a 9
4               # con passo 2: [3, 5, 7, 9]
```

^aAnche in questo caso si ha l'estremo destro escluso.

Esempio 12. Un modo più comodo per ciclare sui tipi dati iterabili (come liste, stringhe e tuple), senza usare `range`, è il seguente:

```
1 for elemento in mia_lista:
2     # ogni elemento è preso in modo ordinato dalla lista
   ↪ mylist
3     print(elemento)
```

Questo modo di ciclare è detto “pythonico”, perché sfrutta appieno le potenzialità del linguaggio. Essere “pythonici” vuol dire proprio questo: scrivere codice che sfrutta quello che ci mette a disposizione il linguaggio evitando di fare le cose a mano quando non serve.

Funzione 4 (Enumerate). La funzione built-in `enumerate` ritorna sia l'elemento che il suo indice nella lista, sotto forma di *tupla*:

```
1 for (i, elemento) in enumerate(mia_lista):
2     print("Posizione {}: elemento {}".format(i, elemento))
```

3 Funzioni

Le *funzioni* in Python sono dichiarate usando la parola chiave `def` e indicando eventuali parametri di input tra parentesi. Dopo i due punti che terminano la prima riga della definizione, si passa poi a un blocco indentato, come già visto sia per le istruzioni condizionali che per i cicli. La parola chiave `return`, alla fine del blocco di istruzioni, precede i valori da ritornare al chiamante. Non è necessario, al contrario di C e C++, indicare il tipo dati degli input, ne tanto meno quello dei valori da ritornare.

Esempio 13. Funzione con due argomenti e nessun ritorno. Se eseguo il codice verrà stampato a schermo “Argomenti: Pippo e Pluto”:

```
1 def mia_funzione(argomento1, argomento2):
2     print(f"Argomenti: {argomento1} e {argomento2}")
3
4 print(mia_funzione('Pippo', 'Pluto')) # Chiamata della
   ↪ funzione
```

Esempio 14. Funzione con un argomento e un ritorno. Se eseguo il codice verrà stampato a schermo “Risultato: 16”:

```
1 def eleva_al_quadrato(numero):
2     return numero * numero
3
4 risultato = eleva_al_quadrato(4)
5 print(f"Risultato: {risultato}")
```

Python dispone di molte funzioni *built-in*, ovvero disponibili senza dover importare nessuna libreria. È possibile consultarne un elenco [qui](#).

3.1 Scope

Si può pensare ad uno *scope* come ad una regione di codice. In Python si stabilisce una gerarchia tra diversi scope. Una variabile, infatti, è disponibile solo all'interno dello scope in cui è stata creata e in tutti gli scope di livello più basso nella gerarchia. È importante saper prevedere quali variabili apparterranno a quali scope, in modo da non far confusione con il loro utilizzo. Python segue la regola *LEGB*, che indica la seguente gerarchia:

- Local: scope locale relativo a ciascuna classe e funzione (ogni classe e funzione ha il suo local scope). È il livello più basso nella gerarchia.
- Enclosed: scope che racchiude la regione di codice attuale, come ad esempio la funzione esterna nel caso di due funzioni annidate, o più semplicemente anche il "corpo" esterno del programma in cui sto definendo una funzione.
- Global: scope più esterno, contiene i nomi visibili in tutto il codice. È sconsigliato l'utilizzo diretto di variabili globali all'interno di funzioni, in quanto si può perdere in interpretabilità e creare bug indesiderati.
- Built-in: scope che contiene i nomi riservati ai moduli built-in di Python. È importante non sovrascrivere i nomi built-in per evitare malfunzionamenti nel codice ¹.

¹Ad esempio, mai chiamare una variabile `sum`, perchè in Python esiste una funzione built-in `sum` che verrebbe in questo modo sovrascritta.

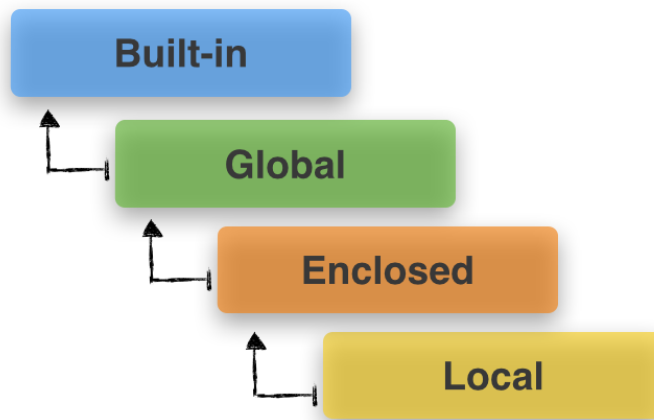


Figura 1. Struttura LEGB

È buona norma, in una funzione, utilizzare variabili locali. Si noti che un argomento in input è considerato locale dalla funzione, in quanto il suo scope è la funzione stessa a cui viene passato. Per salvare un risultato, è prassi passarlo ad una variabile dello scope superiore usando **return**, mentre è sconsigliato modificare variabili dello scope superiore direttamente dall'interno della funzione. Una funzione dovrebbe essere vista come un elemento isolato, che lavora solo sulle variabili locali e comunica con lo scope esterno solo tramite l'input (i suoi argomenti) e l'output (ciò che segue il return).

3.2 Moduli

Un *modulo* è un file che funge da libreria di funzioni. Le funzioni presenti in un modulo possono essere *importate*, usando la parola chiave **import**, ossia **import modulo**, nel file principale. Di norma, per utilizzare una funzione di un determinato modulo, è necessario precedere il nome della funzione con il nome del modulo importato. Alternativamente, si può importare direttamente una singola funzione con la sintassi **from modulo import funzione**. In quest'ultimo caso, il nome della funzione non deve essere preceduto da quello del modulo. I moduli non verranno usati durante il corso.

Esempio 15. Due modalità diverse per usare la stessa funzione **sqrt** del modulo **math**:

```
1 import math
2 var = math.sqrt(4) # var = 2
3
4 from math import sqrt
5 var = sqrt(4)      # var = 2
```

4 Essere pythonici

Quando si scrive codice Python, in particolare se si arriva da altri linguaggi di programmazione, si potrebbe essere portati ad adottare una filosofia “classica” nella scrittura del codice.

Per esempio, per iterare su di una lista si potrebbe essere portati ad usare gli indici:

Esempio 16. Iterazione su di una lista usando un indice di supporto

```
1 mia_lista=['a','b','c']
2 i=0
3 while (i < len(mia_lista)):
4     print(mia_lista[i])
5     i = i + 1
```

Il codice qui sopra riportato funziona, ma non è *pythonico*. Essere Pythonici vuol dire sfruttare appieno le potenzialità del linguaggio, che porta ad una sintassi più semplice e un codice più compatto ed efficiente.

Con Python si può infatti quasi rasentare lo pseudocodice, e programmare in modo quasi discorsivo. Vediamo come possiamo iterare su di una lista in un modo più pythonico, riprendendo quanto già esposto nelle sezioni precedenti:

Esempio 17. Iterazione su di una lista in modo pythonico

```
1 mia_lista=['a','b','c']
2 for elemento in mia_lista:
3     print(elemento)
```

Ci sono molti trucchetti per scrivere codice in modalità più pythonica, e in generale laddove ci si ritrova ad usare degli indici o del codice poco facilmente comprensibile, questo è segno che c'è qualcosa di ampiamente migliorabile.

È una capacità che si fa propria con gli anni, ma che è bene inquadrare fin da subito, poichè facilita notevolmente nella programmazione con Python. Su <https://coady.github.io> si può trovare un blog con vari spunti su come essere più pythonici (in inglese).

5 File e database

I file sono sequenze di bit dove vengono salvate informazioni. Possono essere di due tipologie primarie, ovvero di testo semplice (*plain text*) oppure binari. Un codice sorgente in C, così come un listato Python, è un testo semplice, mentre un codice sorgente C compilato in formato macchina è un file binario. I file di testo semplice si possono editare sia nella Shell tramite interfacce testuali, che con programmi ad interfaccia grafica come ad esempio un generico “blocco note” (Notepad sui sistemi Windows e TextEdit sui sistemi Apple). I file di testo semplice sono in realtà nati per immagazzinare documenti di testo, e agli inizi dell’informatica erano l’unico formato supportato per il testo (da qui, l’estensione `.txt`). Successivamente sono poi stati introdotti formati di testo che supportavano la formattazione, come il grassetto, il corsivo, i paragrafi, le tabelle etc. Questi possono essere sempre dei file di testo semplice arricchiti con dei “tag” secondo delle logiche chiamate di Markup (l’HTML ricade in questa categoria) oppure in formato binario, come i file di Microsoft Word (`.doc`).

Per salvare dei dati strutturati, come ad esempio dei dati temporali e numerici, si potrebbe semplicemente far ricorso a dei documenti di testo, indifferente-mente se di testo semplice o più sofisticati. Potremmo per esempio salvare in una tabella di un documento Word i valori di una determinata azione in borsa giorno per giorno, oppure in un file di testo semplice annotare il prezzo e la data e andare a capo per ogni settimana.

Consultare dei dati in questa modalità sarebbe però estremamente scomodo, se non per quantità veramente limitate. Per semplificare il salvataggio e il successivo recupero di dati strutturati sono stati allora inventate le basi di dati (*Database*, in inglese). I database forniscono un modo estremamente completo per salvare dati e poi consultarli, e sono ad oggi disponibili in varie tipologie, a seconda del contesto in cui vengono utilizzati. La creazione e il mantenimento di un database hanno però bisogno di un attento lavoro di design e gestione delle risorse, che può risultare eccessivo per compiti semplici.

Il formato CSV Un *file* in formato *CSV*, ovvero “Comma-Separated Values”, è un buon compromesso tra l’utilizzo di un database vero e proprio e il salvataggio dei dati in un documento di testo. In un CSV, ogni riga è un’entrata (“entry” in inglese) con valori separati da una virgola, che formano le colonne. È possibile inserire un’intestazione (opzionale) per dare un nome alle colonne. L’estensione di questi file è `.csv`, o anche semplicemente `.txt` in quanto sono a tutti gli effetti documenti di testo semplice. Durante il corso verrà usato il file `shampoo_sales.csv`, che riporta le vendite di shampoo per un periodo di tre anni di cui qui sotto si riporta una anteprima e il cui andamento è mostrato graficamente in [Figura 2](#).

```
Date,Sales
01-01-2012,266.0
01-02-2012,145.9
01-03-2012,183.1
01-04-2012,119.3
...
```

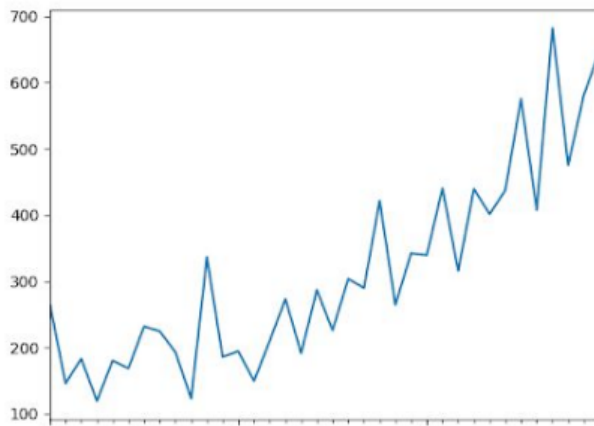


Figura 2. Andamento dei valori contenuti nel file `shampoo_sales.csv`

I files in Python Python mette a disposizione varie modalità per interagire con i files, tutte disponibili in seguito ad aver aperto il file tramite la funzione built-in `open()`, che apre il file passato come argomento ¹.

Funzione 5 (`open`). Per aprire un file in Python si usa la funzione built-in `open()`, che ritorna un'entità di tipo `file`. Gli argomenti da passare alla funzione sono il nome del file e una stringa per indicare se si vuole aprire il file in modalità di lettura, scrittura, aggiunta o combinazioni di queste. In particolare:

- `'r'` indica modalità *read*;
- `'w'` indica modalità *write*, si sovrascrive file;
- `'rw'` indica modalità mista *read* e *write*;
- `'a'` indica modalità *append*, in cui aggiungo alla fine del file.

Funzione 6 (`file.close`). Per chiudere un file si usa la funzione dell'entità `file` `close()`. È necessario chiudere un file per liberare memoria e per salvare alcuni cambiamenti che potrebbero essere ancora non stati scritti sul disco.

Funzione 7 (`file.read`). La funzione `read()` legge l'intero file e ne ritorna il contenuto.

¹Se il file non si trova nella stessa directory del codice, si deve indicare il percorso da seguire per trovarlo, relativamente alla directory in cui ci si trova o assoluto.

Funzione 8 (`file.readline`). La funzione `readline()` legge una riga per volta e ne torna il contenuto. Chiamando `readline()` più volte sulla stessa entità `file` si andrà automaticamente alle righe successive.

Esempio 18. Il seguente snippet di codice apre il file in modalità lettura, ne stampa i primi 50 caratteri usando lo *slicing* delle stringhe e lo chiude:

```
1 mio_file = open('shampoo_sales.csv', 'r')
2 print(mio_file.read()[0:50])
3 mio_file.close()
```

Esempio 19. Il seguente snippet invece, apre il file in modalità lettura, legge e stampa le prime 5 righe e lo chiude:

```
1 mio_file = open('shampoo_sales.csv', 'r')
2 for i in range(5):
3     print(mio_file.readline())
4 mio_file.close()
```

Funzione 9 (`file.write`). Per scrivere su file, dopo averlo aperto in modalità scrittura (`'w'`), si usa la funzione dell'entità `file` `write()`.

Per leggere tutte le righe di un file, si può sia caricarne tutto il contenuto in memoria con la funzione `read()` e poi trovare un modo di processarlo riga per riga, o in alternativa si può usare la funzione `readline()` ripetutamente, finché il valore tornato non sarà `None`, a significare che non ci sono più nuove righe. Tuttavia, il modo pythonico di leggere un file riga per riga è più semplice, e va a sfruttare l'iterabilità dell'entità `file`, come esposto nell'esempio seguente:

Esempio 20. Un file letto riga per riga in modo pythonico

```
1 mio_file = open('shampoo_sales.csv', 'r')
2 for linea in mio_file:
3     print(linea)
4 mio_file.close()
```

Si noti che Python “sa” che l'unità dell'iterazione sono le righe: riconosce il carattere che indica l'andare a capo e lo usa come criterio per passare ad una nuova iterazione.

Un'altra pratica pythonica è di aprire un file non in maniera a sè stante come fatto sopra, ma all'interno di un cosiddetto *contesto di esecuzione*, che si apre tramite la funzione built-in `with()`. Questo è un blocco di codice che

fa delle cose al posto nostro per aiutarci e in particolare laddove lo usiamo per interagire con un file, chiude lui per noi il file. In pratica, non dovremo chiamare la chiusura del file a mano: lo farà già il contesto di esecuzione per noi. Possiamo sempre usare questo “aiutante” per leggere i file.

Esempio 21. Un file letto all'interno di un contesto di esecuzione, riga per riga in modo pythonico

```
1 with open('shampoo_sales.csv', 'r') as mio_file:
2     for linea in mio_file:
3         print(linea)
```

Leggere un file CSV Python mette a disposizione un modulo per la lettura di file CSV, ma a fini didattici scriveremo noi il codice necessario. Per prima cosa dobbiamo introdurre alcune nuove funzioni, e in particolare la funzione `split()` dell'entità stringa.

Funzione 10 (`str.split`). La funzione `split()` dell'entità stringa divide la stringa quando incontra il carattere indicato come argomento tra parentesi e ritorna una lista contenente i vari "pezzi".

Esempio 22. Un esempio di come dividere una stringa in base ad uno spazio come divisore dei pezzi. La variabile `pezzi` è una lista.

```
1 mia_stringa = 'Ciao mondo'
2 pezzi = mia_stringa.split(' ') # pezzi = ['Ciao', 'mondo']
```

Divideremo quindi ogni riga del file in base al separatore virgola. quello che viene letto da un file CSV (che ricordiamo è salvato come testo semplice) sono sempre stringhe, anche se rappresentano informazioni numeriche. Per farne uso sarà quindi necessaria una conversione, che ci introduce al concetto di *casting* di tipi dati.

Sintassi 10 (Casting). Per convertire valori tra diversi tipi dati si usa la sintassi di casting, in cui si usa direttamente il tipo dati di destinazione, passandoci il valore (o variabile) da convertire tra parentesi:

```
1 float('9.1') # Da stringa numerica a floating point: 9.1
2 int(9.1)      # Da floating point a intero: 9
3 float(9)      # Da intero a floating point: 9.0
4 str(9)        # Da intero a stringa: '9'
5 float('hey')  # Da stringa testuale a floating point: errore
6 float([])     # Da lista a floating point: errore
```

Se il tipo dati di destinazione non può rappresentare il valore originale, que-

sto può essere troncato perdendo informazione (come per la conversione da floating point a intero), oppure può venire modificato anche aggiungendone di nuova (come per la conversione da intero a floating point, in cui si aggiunge una cifra decimale, posta a zero). Se invece il tipo dati di destinazione non può in nessun modo rappresentare l'originale (come ad esempio cercando di convertire una lista a floating point), allora verrà generato un errore.

Una volta divisi i valori di ogni linea del file in corrispondenza della virgola e convertiti quelli numerici a intero o a floating point, dobbiamo aggiungerli ad una struttura dati adatta per immagazzinarli tutti. Per semplicità scegliamo la lista, in cui ogni elemento sarà a sua volta una lista con un elemento per ogni valore trovato nella riga del file CSV: avremo quindi una *lista di liste*.

Un primo approccio, spesso seguito in C, sarebbe quello di pre-allocare le liste in base a quante righe ed elementi ha il file. Questo comporta dover sapere a priori quanti elementi saranno presenti, e ciò non è sempre possibile, nonché scomodo e poco flessibile. Possiamo invece aggiungere dati ad una lista già esistente, compresa la lista vuota, man mano che li processiamo, utilizzando la funzionalità di `append` delle liste.

Funzione 11 (`list.append`). La funzione `append()` dell'entità lista aggiunge un elemento alla fine della lista, modificandone la lunghezza:

```
1 mia_lista = []
2 mia_lista.append(1) # mia_lista = [1]
3 mia_lista.append(2) # mia_lista = [1, 2]
```

Abbiamo ora tutti gli strumenti per leggere informazioni da un file CSV e tornarli sotto forma di lista, vediamo come:

Esempio 23. Nel seguente esempio, si legge da `shampoo_sales.csv` e si popola la lista `dati` che ne conterrà le informazioni:

```
1 # Inizializzo una lista vuota per salvare i dati
2 dati = []
3 # Apro e leggo il file, linea per linea
4 with open( 'shampoo_sales.csv' , 'r' ) as mio_file:
5     for linea in mio_file:
6         # Faccio lo split di ogni riga sulla virgola
7         elementi = linea.split(',')
8         # Se NON sto processando l'intestazione...
9         if elementi[0] != 'Date':
10             # Setto le variabili data e il valore,
11             # convertendo il valore a floating point
12             data = elementi[0]
```

```
13     valore = float(elementi[1])  
14     # Aggiungo alla lista dei dati  
15     dati.append([data,valore])
```

Programmazione ad oggetti

PART

III

6 Programmazione ad oggetti

6.1 Classi ed oggetti

La **Programmazione orientata agli Oggetti** (in inglese *Object-Oriented Programming, OOP*), o semplicemente **Programmazione a oggetti** è un paradigma di programmazione, a cui **Python** appartiene, che permette di ragionare in termini di entità e non più solo di funzioni con input/output. Queste entità prendono il nome di *oggetti* e hanno due grossi vantaggi:

1. possono mantenere uno stato, e
2. sono organizzabili gerarchicamente.

Gli oggetti si definiscono tramite le *classi*. Una classe è una generica definizione di un oggetto, ovvero uno schema che permette di descrivere un'entità con le caratteristiche desiderate. Ad esempio, una persona con un nome e un cognome. Una specifica entità è invece chiamata *istanza*, come ad esempio la persona specifica Mario Rossi. L'operazione di passare da classe ad istanza di oggetto si chiama *istanziamento*. Il concetto di oggetto è generico e racchiude sia classi che istanze, è un po' una parola "jolly" per esprimere un concetto o l'altro a seconda dei contesti. In genere che si parli di classe o istanza è infatti abbastanza evidente, ma nei casi dubbi è sempre meglio specificare a cosa ci si sta riferendo, sia nei commenti del codice che nel descriverlo o discuterlo. Non c'è un limite a quante istanze di classe si possono creare, e possono essere create anche dinamicamente e salvate in strutture di supporto come ad esempio le liste.

Una classe in **Python** rappresenta, in modo abbastanza intuitivo, l'equivalente della **struct** in **C**. In particolare, come la **struct** serve a definire un tipo, raccogliendo quindi la definizione delle strutture dati (campi, o valori) che rappresentano l'entità concettuale che vogliamo modellare - ciascuna istanza della classe rappresenta una realizzazione indipendente del concetto modellato dalla classe. A differenza della **struct**, invece, la classe non raccoglie solamente valori, ma anche funzioni - o metodo della classe - per operare sulle entità che modelliamo. Le funzioni contenute in una classe prendono il nome di *metodi*, mentre le variabili prendono il nome di *attributi*. Sono sempre funzioni e variabili, ma in questo contesto cambiano nome. Non è un errore parlare di funzioni e variabili nel contesto della programmazione ad oggetti, ma è piuttosto una imprecisione.

Le classi permettono quindi di definire in modo astratto ma intuitivo delle entità con caratteristiche ben definite. Ad esempio, una classe **Persona** può avere un attributo **nome** e un metodo **saluta()**. Se si volesse salvare il nome di una persona senza usare l'attributo di una classe lo si dovrebbe salvare in una struttura a parte, come una lista o un dizionario. Questo può risultare scomodo, in quanto è necessario tenere a mente quali strutture hanno quale compito. In più, gli indici per accedere ai vari elementi nelle liste potrebbero velocemente diventare troppi. Quindi, le classi offrono una soluzione semplice per salvare in modo compatto le informazioni legate ad un'entità logica, come una persona.

Altri linguaggi di programmazioni ad oggetti, quali **Java** o **C++**, mantengono il concetto di classe, seppur con lievi differenze legate prevalentemente alla parte imperativa del linguaggi (ad esempio, in **C++** ci sono i puntatori espliciti, che in **Java** sono solamente impliciti).

Python e gli oggetti In Python **tutto**, tranne le parole chiave, è un **oggetto**: numeri, stringhe, liste, persino le funzioni e, ovviamente, gli oggetti stessi. Per esempio, fino ad ora abbiamo parlato di “entità stringa”, e della funzione **split()** dell'entità. In realtà la stringa è un oggetto, e **split()** è un suo metodo (che ne fa la divisione in pezzi in base al separatore specificato come argomento). Similmente, abbiamo parlato di “entità file”, ma in realtà quello che ci ritorna la funzione built-in **open()** è proprio un oggetto di tipo **file**. E quando, per leggere un file, abbiamo invocato la “funzione **read()**”, abbiamo in realtà usato un metodo dell'oggetto file (che ci torna il contenuto del file rappresentato dal suo oggetto).

Per convincerci che in Python è tutto un oggetto e controllarne il tipo, possiamo usare le funzioni built-in **dir()** e **type()**.

Funzione 12 (**dir**). La funzione **dir()** ritorna una lista con tutti gli attributi e i metodi di un oggetto

Esempio 24. La funzione **dir()** applicata ad una stringa. Si può vedere listato, tra gli altri, anche il metodo **split()**.

```
1 >>> dir('ciao')
2 ['__add__', '__class__', '__contains__', '__delattr__',
3 ... 'split', 'splitlines', 'startswith', 'strip', ...
4 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

Funzione 13 (**type**). La funzione **type()** ritorna la classe dell'oggetto. Poiché in Python tutto è un oggetto, la stessa definizione di “tipo” passa per il concetto di classe.

Esempio 25. La funzione `type()` applicata ad un numero, ad una stringa, e ad una funzione.

```
1 >>> type(3.14)
2 <class float>
3 >>> type('ciao')
4 <class str>
5 >>> type(print)
6 <class builtin_function_or_method>
```

Come già anticipato sopra, tutto in Python, tranne le parole chiave, è un'istanza di una classe ovvero un oggetto, ed è sempre possibile esplorarne le caratteristiche con le funzioni appena presentate.

Quando per esempio si converte da un tipo dati ad un altro, con una sintassi del tipo `float('3.14')`, si sta creando una nuova istanza dell'oggetto `float` a partire da una stringa (dal contenuto 3.14).

Per controllare se una variabile è istanza di un determinato tipo di classe (ovvero se un oggetto è di un certo tipo), si può usare la funzione built-in `isinstance()`:

Funzione 14 (`isinstance`). La funzione `isinstance()` verifica se il suo primo argomento è istanza della classe specificata dal secondo argomento.

```
1 print(isinstance(variabile, MiaClasse))
2 # True se variabile è istanza di MiaClasse, False altrimenti
```

Operazioni in-place Quando si chiama un metodo di un oggetto, a differenza delle funzioni a sé stanti, questo può agire direttamente sull'oggetto senza tornare niente. Non è una pratica consigliata, in quanto dovrebbe sempre valere il paradigma `input -> computazione -> output`, ma è una pratica accettata laddove ci siano ragioni specifiche, come per esempio di performance. Nel caso un metodo modifichi direttamente un oggetto non tornando niente (`None`), questa operazione prende il nome di *in-place*, ovvero che viene eseguita sul posto. Se invece il metodo non modifica lo stato dell'oggetto allora non parlerò di operazione in-place. Bisogna stare attenti con le operazioni in-place perché possono portare a non accorgersi di star cambiando gli oggetti originali, causando bug e mal di testa.

Esempio 26. Un'operazione in-place e una non in-place:

```
1 mia_lista = [1,2,3]
2 print(mia_lista.reverse()) # Stampa "None"
3 print(mia_lista)          # Stampa "[3,2,1]",
4                             # mia_lista è stata modificata
```

```

5
6 mia_stringa = 'ciao mondo'
7 print(mia_stringa.upper()) # Stampa "CIAO MONDO".
8 print(mia_stringa)         # Stampa "ciao mondo",
9                             # mia_stringa è rimasta
                             ↪ invariata.

```

Un'ultima questione importante nell'approcciare la programmazione ad oggetti in Python è la notazione, in termini di formato, da usare nel codice. Di norma, si usa il *camelcase* (es. `MyClass`) per i nomi delle classi e lo *snake-case* (es. `my_method`) per i nomi dei metodi e delle variabili. Ci sono alcune eccezioni a questa convenzione, come ad esempio tutti i tipi dati built-in (parliamo di `float` e non di `Float`). Negli altri casi, è sempre buona norma seguire questa convenzione poiché altrimenti si perderebbe di interpretabilità del codice.

Definire una classe La sintassi per definire una classe, nel caso più semplice possibile, è la seguente:

```

1 class class_name:
2     docstring
3
4     .. variabili ..
5
6     .. funzioni ..

```

dove:

- `class` è la parola chiave per definire la classe in Python
- `class_name` è il nome della classe;
- `variabili` rappresentano variabili di istanza di ciascun oggetto che viene costruito;
- `funzioni` rappresentano funzioni associate (cioè implementate) per tutti gli oggetti della classe.

Ad esempio

```

1 class lamiaclassa:
2     "Ho creato una classe"
3
4     # funzione definita all'interno della classe
5     def fun(self):
6         print("Sto stampando da dentro la classe")

```

Nota bene (`self`). Il parametro `self` nel metodo `fun` è una variabile che indica l'istanza corrente della classe, ovvero l'oggetto specifico su cui il metodo `fun` viene invocato. Questo `fun` in realtà non usa mai `self`, ma il passaggio di `self` a `fun` è obbligatoria. In particolare, nella definizione dei metodi delle classi, usare `self` come primo argomento è obbligatorio, mentre nella loro chiamata va omissso. Il concetto espresso da `self` è più o meno presente in tutti i linguaggi ad oggetti, ad esempio in Java il `self` non è altro che il `this`.

Proviamo a mettere in pratica la definizione di classe per descrivere gli studenti dell'Università di Trieste.

```
1 class Studente:
2     # attributo uguale per tutti
3     ruolo = "Studente UNITS"
4
5     # costruttore, per ogni studente,
6     # che permette di creare un oggetto
7     # della classe, con opportuni valori per
8     # eventuali variabili di istanza dell'oggetto Studente
9     def __init__(self, nome, cognome):
10         self.nome = nome
11         self.cognome = cognome
12
13     def saluta(self):
14         print('Ciao sono', self.ruolo + ", ", self.nome,
              ↪ self.cognome)
```

Nel caso particolare un oggetto di classe `Studente` ha 3 attributi:

- `nome` e `cognome`, che vengono settati su `self` all'interno di `__init__`;
- `ruolo`, un attributo uguale per tutte le istanze della classe `Studente`.

Questo esempio ci chiarisce cosa significhi creare un oggetto di una classe. Quando viene creato un oggetto di una certa classe, si istanzia un particolare elemento della classe di appartenenza. Questo avviene attraverso il metodo `__init__`, ovvero il metodo *costruttore* (l'*inizializzatore*) della classe. Per accedere agli attributi all'interno della classe, ovvero nel suo corpo di codice, si usa sempre la parola chiave `self`, che rappresenta l'istanza, seguita dal nome dell'attributo (o metodo) cui si vuole accedere, separati da un punto.

Per accedere agli attributi (o i metodi) dell'istanza della classe, si usa il nome assegnato alla variabile che "contiene" l'istanza della classe, seguita da un punto e dal nome dell'attributo (o metodo) cui si vuole accedere.

Ad esempio, le variabile `studente1` sarà un'istanza della classe `Studente`, così come `studente2`.


```

1 studente1 = Studente('Mario', 'Rossi')
2 studente2 = Studente('Irene', 'Bianchi')
3 print(studente1.nome) # 'Mario'
4 print(studente2.nome) # 'Irene'
5 persona1.saluta()     # 'Ciao sono Studente UNITS, Mario, Rossi'
6 persona2.saluta()     # 'Ciao sono Studente UNITS, Irene Bianchi'

```

`__init__` è un particolare tipo di metodo che appartiene ai cosiddetti *metodi magici* (magic methods o dunder).

Nota bene (Magic methods). I magic methods sono particolari metodi implementabili nelle classi Python, che permettono di far sì che le nostre classi possano interagire con alcuni operatori e costrutti del linguaggio. Tutti i cosiddetti magic methods hanno nome in forma `__method__` caratterizzata da underscore doppi (`__`). Di norma, i metodi con il doppio underscore seguono una convenzione secondo cui sono di uso tecnico interno, privato, e non dovrebbero essere invocati esplicitamente.

Esempio 27 (String method). Il metodo magico `__str__` si usa per creare una rappresentazione in stringa dell'oggetto di una classe. Tale metodo viene invocato ogni volta che si chiama `print()` su un oggetto della classe.

```

1 class Studente:
2     ruolo = "Studente UNITS"
3
4     def __init__(self, nome, cognome):
5         self.nome = nome
6         self.cognome = cognome
7
8     def saluta(self):
9         print('Ciao sono', self.ruolo + ", ", self.nome,
10             ↪ self.cognome)
11
12 studente1 = Studente('Mario', 'Rossi')
13
14 print(studente1) # <__main__.Studente object at 0x104e295d0>

```

Posso sovrascrivere `__str__` scrivendo che stringa deve essere stampata.

```

1 class Studente:
2     ruolo = "Studente UNITS"
3
4     def __init__(self, nome, cognome):
5         self.nome = nome
6         self.cognome = cognome

```

```

7
8     def saluta(self):
9         print('Ciao sono', self.ruolo + ",", self.nome,
10             ↪ self.cognome)
11
12     def __str__(self):
13         return 'Persona "{} {}".format(self.nome,
14             ↪ self.cognome)'
15
16 studente1 = Studente('Mario', 'Rossi')
17
18 print(studente1) # Persona "Mario Rossi"

```

Quindi `__init__` permette di creare lo stato interno di una particolare istanza di un oggetto, un'operazione che sostanzialmente comporta l'assegnamento di un valore agli attributi che caratterizzano l'oggetto stesso. In questo caso quindi ogni oggetto di tipo `Studente` viene creato con gli attributi `nome`, `cognome`, `ruolo`. Si noti quindi che ogni oggetto avrà i suoi valori di `nome` e `cognome`, ma invece `ruolo` risulterà costante per tutti gli oggetti della classe.

Si noti che stavolta il metodo `saluta`, ricevendo in input un particolare oggetto, utilizza `self.ruolo` e `self.nome` etc. per accedere al valore degli attributi dell'oggetto (indipendentemente dal fatto che siano stati creati nel costruttore oppure no).

Esempio 28. Utilizzo di oggetti come componenti di una lista

```

1 studenti = [ Studente('Mario', 'Rossi'),
2             Studente('Irene', 'Bianchi')]
3 print(studenti[0].nome) # Mario
4 studenti[1].saluta()    # 'Ciao sono Studente UNITS, Irene,
   ↪ Bianchi'

```

Esercizio 6.1. Non affrontiamo un discorso sulla memoria in Python. Tuttavia, si ragioni ad alto livello su come si rappresenterebbe la memoria di un programma che crea oggetti della classe `Studente`.

Si noti che i vincoli descritti precedentemente riguardanti il parametro `self`, si applicano anche al costruttore della classe (ed agli altri metodi magici).

Si noti anche che i campi di un oggetto così definiti sono modificabili (come nelle `struct` in C).

```

1 studente1 = Studente('Mario', 'Rossi')
2 studente2 = Studente('Irene', 'Bianchi')
3 studente1.nome = "Laura" # cambio nome

```

```

4 print(studente1.nome) # Laura
5
6 studente1.ruolo = "Bidello" # cambio ruolo
7 studente1.saluta() # Ciao sono Bidello, Laura Rossi
8 studente2.saluta() # Ciao sono Studente UNITS, Irene Bianchi

```

Si provi ad inserire il codice qui sopra per osservare il comportamento dell'interprete Python. Notare che la modifica di un attributo di classe (definito fuori del costruttore) su una istanza modifica l'attributo solo su quell'istanza. Se voglio modificare l'attributo per tutte le istanze dovrò accedere tramite il nome della classe.

```

1 studente1 = Studente('Mario', 'Rossi')
2 studente2 = Studente('Irene', 'Bianchi')
3 studente1.nome = "Laura" # cambio nome
4 print(studente1.nome) # Laura
5
6
7 Studente.ruolo = "Bidello" # cambio ruolo
8 studente1.saluta() # Ciao sono Bidello, Laura Rossi
9 studente2.saluta() # Ciao sono Bidello, Irene Bianchi

```

Esercizio 6.2 (Modellazione a classi). Si elenchino almeno 5 contesti di applicazione per la programmazione, in cui potremmo beneficiare di introdurre delle classi, e si discuta di cosa modellerebbero i relativi oggetti. Per esempio: software di gestione di un zoo, in cui ogni animale è rappresentato dalla sua classe, e gli oggetti sono gli specifici animali.

6.2 Ereditarietà tra classi

L'**ereditarietà** è uno dei concetti fondamentali nel paradigma di programmazione a oggetti, e consiste in una relazione che il programmatore stabilisce tra due classi.

Se la classe B eredita dalla classe A, si dice che B è una *sotto-classe* di A e che A è una *super-classe* di B. A seconda del linguaggio di programmazione, l'ereditarietà può essere singola (ogni classe può avere al più una super-classe) o multipla (ogni classe può avere più super-classi). In Python entrambe sono possibili, ma nel nostro caso ci limiteremo a discutere l'ereditarietà singola. Si noti inoltre che la relazione è transitiva: in un programma potrei avere A super-classe di B, e B super-classe di C.

Nota bene (Interpretazione modellistica dell'ereditarietà). L'ereditarietà è una relazione di generalizzazione e specializzazione: la super-classe definisce un concetto generale e la sotto-classe rappresenta una variante specifica di tale concetto generale. Su questa interpretazione si basa tutta la teoria dell'ereditarietà nei linguaggi a oggetti. Oltre a essere

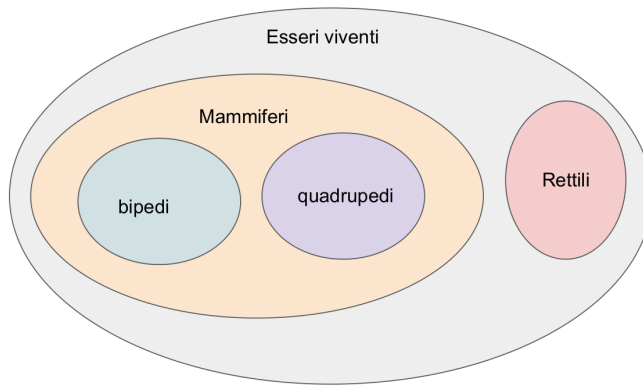


Figura 3. Esempio di gerarchie tra classi in termini di insiemi e relazioni di inclusione

un importante strumento di modellazione, l’ereditarietà ha importanti ricadute sulla riusabilità ed il design del software.

Quando una sotto-classe eredita da una super-classe, tutti gli attributi e i metodi di istanza della super-classe sono automaticamente presenti nella sotto-classe. Per questo motivo, il termine *ereditarietà*: la sotto-classe eredita almeno il comportamento (variabili e metodi) della super-classe. Proprio per questo motivo la super-classe è una classe generale - almeno più generale della sotto-classe! - che serve a definire gli attributi comuni ad una serie di classi specializzate.

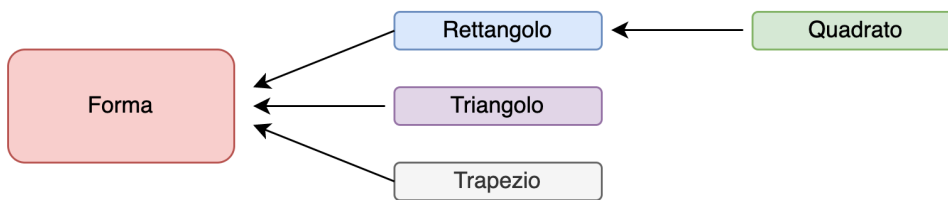
Esempio 29. Supponiamo di voler modellare una gerarchia di concetti astratti quali gli “animali”. Tutti i “mammiferi” sono animali, ma non tutti gli animali sono mammiferi. Quindi, se dovessimo modellare con delle classi gli oggetti mammifero e animale, allora sicuramente animale sarebbe la super-classe di mammifero. Possiamo interpretare le gerarchie tra classi in termini di insiemi e relazioni di inclusione, come mostrato in Figura 3.

Non esiste un metodo “corretto” di rappresentare le gerarchie, dipende dal contesto. Pensiamo per esempio ai bicchieri: si potrebbe creare una gerarchia tra classi dove una generica classe **Bicchiere** viene estesa da due sottoclassi **BicchiereAlcolico** e **BicchiereAnalcolico**, oppure da **BicchiereVetro** e **BicchierePlastica**. La prima è probabilmente più adatta nel contesto della ristorazione, la seconda nella logistica e nei magazzini.

Esempio 30. In maniera simile, potremmo pensare di modellare le forme geometriche, suddivise in quadrati, rettangoli, triangoli etc., oppure il personale di una università (Figura 4).

Continuando l’esempio della costruzione di un programma che modelli alcuni dei ruoli lavorativi esistenti all’interno dell’università. Chiediamoci, cosa accomuna tutte le persone che vorremmo descrivere? Senza alcuna presunzione,

a



b

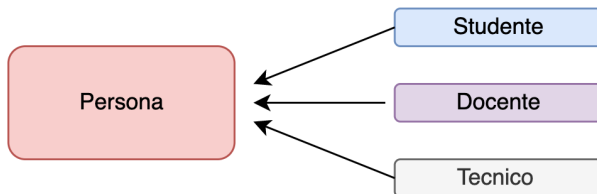


Figura 4. a) Esempio di relazione di ereditarietà. Un **Rettangolo**, un **Triangolo** ed un **Trapezio** sono sotto-classi di una generica classe **Forma** (super-classe). Inoltre, un **Quadrato** è una sotto-classe di un **Rettangolo** - perchè è un rettangolo con la particolarità di avere tutti i lati uguali. b) Con un simile ragionamento possiamo intuire che una ipotetica classe **Persona** sia la super-classe per classi specializzate quali uno **Studente**, un **Docente** o un **Tecnico**.

sicuramente tutti coloro che vogliamo descrivere sono delle persone ed hanno un’“anagrafica”, che è indipendente dallo specifico ruolo ricoperto. Senza tergiversare tanto su cosa sia o meno opportuno come anagrafica - non è questo il punto - se definiamo il concetto di persona tramite una super-classe, allora ad esempio gli studenti, i docenti ed i tecnici universitari sono tutti esempi specializzati (sotto-classi) di una generica *persona*.

Cominciamo in codice a descrivere la super-classe:

```

1  # classe persona con anagrafica (super-classe)
2  class Persona:
3      def __init__(self, ruolo, nome, cognome):
4          self.ruolo = ruolo
5          self.nome = nome
6          self.cognome = cognome
7
8      def saluta(self):
9          print('Ciao sono', self.ruolo + ",", self.nome,
              ↪ self.cognome)
  
```

Esercizio 6.3. In questo caso, rispetto all’esempio precedente, abbiamo associato il ruolo al costruttore di **Persona**, perché?

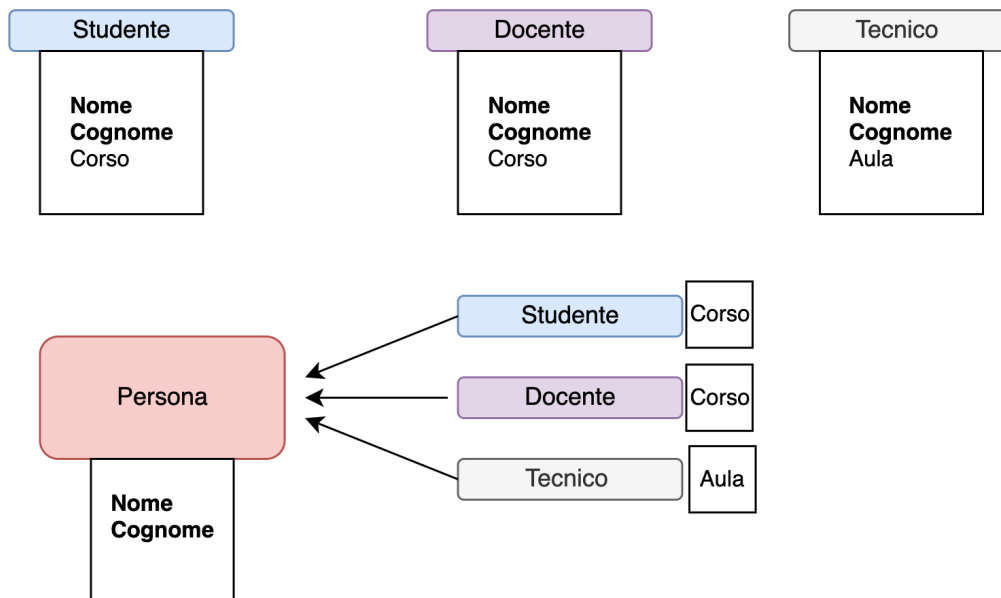


Figura 5. Possiamo automatizzare il processo che porta al punto b di Figura 4. Elenchiamo gli attributi che assegniamo a studenti, docenti e tecnici; supponiamo tutti abbiano una anagrafica (nome e cognome, in grassetto), più degli attributi specifici: il corso seguito da uno studente, il corso insegnato da un docente, oppure l'aula in cui il tecnico opera. A questo punto tutte le variabili di istanza condivise (nome e cognome), ha senso facciano parte del super-classe **Persona**, così che vengano definite una sola volta e siano disponibili in tutte le sotto-classi che estendono **Persona**. Gli altri valori rimangono invece privati alle sotto-classi perché non condivisibili (ad esempio, uno studente non viene associato ad una aula).

A questo punto posso dichiarare una sotto-classe, la sintassi generale richiede di dichiarare

```
1 class nome_classe(nome_super_classe)
2     ....
```

laddove ovviamente `nome_super_classe` rappresenta il nome della super-classe padre di `nome_classe`. Quindi possiamo definire una sotto-classe **Studente**, supponendo di voler memorizzare, oltre all'anagrafica che otteniamo da una generica **Persona**, anche ulteriori informazioni relative almeno ad un corso che uno studente potrebbe seguire (si veda l'esempio in Figura 5).

Riportiamo qui il codice della sotto-classe **Studente**, il quale modella una persona per la quale si ha un corso associato.

```
1 # sotto-classe studente
2 class Studente(Persona):
3
4     # costruttore sotto-classe
```

```

5  def __init__(self, nome, cognome, corso):
6      super().__init__("Studente UNITS", nome, cognome)
7      self.corso = corso
8
9      # ridefinizione del metodo saluta di persona
10     def saluta(self):
11         Persona.saluta(self) # uso esplicitamente metodo di
           ↪ Persona
12         print("> Frequento il corso: ", self.corso)

```

Analizziamo il costruttore della sotto-classe. Ogni studente è anche una persona, perciò i parametri del costruttore sono anche i parametri di un oggetto di classe `Persona`, ovvero `nome`, `cognome` ed un `corso`. Si noti invece che il `ruolo` non è un parametro del costruttore. A questo punto, per costruire uno `Studente` ci avvaliamo del costruttore della classe `Persona`, al quale possiamo passare esattamente i parametri necessari.

```
super().__init__("Studente UNITS", nome, cognome)
```

Successivamente si inizializzano gli attributi specifici di `Studente`. Si noti che per chiamare il costruttore della super-classe – ovvero `__init__` della classe `Persona`, non di `Studente`! – si usa la parola chiave `super()`. Come sarà intuitivo dal nome, `super` non è altro che una funzione che usiamo per specificare che vogliamo usare i metodi disponibili nella super-classe dell'oggetto `self`², e non quindi quelli di `Studente`.

Nota bene. Se si chiama il costruttore della super-classe si devono passare tutti i parametri necessari. Si noti che qui, ad esempio, a `ruolo` associamo una costante `"Studente UNITS"`. Questo perchè nella nostra programmazione non vogliamo lasciare la libertà di cambiare questo valore di ruolo a chi costruisce gli oggetti della classe `Studente`.

Allo stesso modo si possono ridefinire metodi definiti nella super-classe, al cui interno viene chiamato il metodo omonimo della super-classe ed eventualmente vengono aggiunte funzionalità. Questo tipo di comportamento si chiama **overriding** - sovrascrittura - dei metodi della super-classe, e lo vediamo in pratica nella definizione di `saluta`. Per fare overriding si deve ri-definire il metodo della super-classe esattamente con la stessa segnatura

```

1  def saluta(self):
2      ....

```

ma notate che posso decidere di chiamare il metodo della super-classe usando, ancora una volta una sorta di `super()`

²In pratica le classi e le loro gerarchie statiche sono memorizzate in una tabella delle classi. `super()` restituisce un riferimento all'entrata corretta di quella tabella, quella per `Persona`.

```
1      Persona.saluta(self) # uso esplicitamente di un metodo di
      ↪ Persona
```

Un possibile utilizzo del precedente frammento di codice è:

```
1  # creo un oggetto e chiamo la funzione saluta
2  obj_Giulio = Studente("Giulio", "Caravagna", "Programmazione")
3
4  obj_Giulio.saluta()
```

Si verifichi che venga eseguita la funzione `saluta` definita in `Studente`.

In modo analogo si può definire una classe `Docente`:

```
1  class Docente(Persona):
2      def __init__(self, nome, cognome, corso):
3          super().__init__("Docente UNITS", nome, cognome)
4          self.corso = corso
5
6      def saluta(self):
7          Persona.saluta(self)
8          print("> Docente del corso: ", self.corso)
```

Da notare che, per gli attributi che abbiamo definito per `Studente` e `Docente`, la super-classe `Persona` è la più generale che poteva esser definita, poiché entrambe le classi aggiungono solo un attributo a `Persona` (si veda l'esempio di Figura 5).

Possiamo istanziare un `Docente` nel seguente modo:

```
1  obj_Laura = Docente("Laura Nenzi", "Nenzi", "Programmazione")
2  obj_Laura.saluta() # eseguita funzione della sotto-classe
```

Esercizio 6.4 (Modellazione a classi - seconda parte). Con riferimento ai 5 esempi di modellazione a classi elencati nell'Esercizio 6.2, si ragioni ora della possibilità di introdurre delle gerarchie tra le classi elencate. Si discuta anche degli eventuali attributi e metodi che sarebbero di pertinenza delle eventuali super-classi, e delle sotto-classi. Per esempio, continuando l'esempio sullo zoo, gli animali potrebbero essere tutti sotto-classe della super-classe `Animale`, con variabili di istanza che memorizzano l'età o altre caratteristiche comuni a tutti gli animali. I metodi della super-classe potrebbero fornire le operazioni per la gestione – ad esempio `nutrire`, `controllare_salute`, `spostare` etc. – ogni singolo animale dello zoo.

Esempio 31. Si vogliono modificare le classi `Studente` e `Docente` precedentemente definite in modo che esse possano:

- memorizzare una lista di corsi frequentata da uno studente;
- memorizzare una lista di corsi insegnati da un Docente;

Per semplicità le liste in questione non sono vuote. Si vuole quindi definire `saluta` per stampare a schermo la lista completa dei corsi.

In pratica, si vuole permettere di scrivere il seguente codice:

```
1 corsi = ["Programmazione", "Laboratorio", "Analisi",  
2 ↪ "Geometria"]  
obj_Irene = Studente("Irene", "Rossi", corsi)
```

cosicché `obj_Irene.saluta()` stampi tutti i corsi frequentati da `obj_Irene` (variabile `corsi`, passata al costruttore).

Una possibile soluzione è la seguente:

```
1 # sotto-classe Studente con una lista di corsi  
2 # memorizzo anche n, il numero dei corsi seguiti  
3 class Studente(Persona):  
4     def __init__(self, nome, cognome, lista_corsi)  
5         super().__init__("Studente UNITS", nome, cognome)  
6         self.lista_corsi = lista_corsi  
7         self.n = len(lista_corsi)  
8  
9     def saluta(self):  
10        Persona.saluta(self)  
11  
12        # stampo con un for  
13        for i in range(0, self.n):  
14            print("> Frequento il corso: ",  
15                ↪ self.lista_corsi[i])  
16  
17 # sotto-classe Docente con una lista di corsi  
18 # memorizzo anche n, il numero dei corsi insegnati  
19 class Docente(Persona):  
20     def __init__(self, nome, cognome, lista_corsi)  
21         super().__init__("Docente UNITS", nome, cognome)  
22         self.lista_corsi = lista_corsi  
23         self.n = len(lista_corsi)  
24  
25     def saluta(self):  
26        Persona.saluta(self)  
27  
28        # stampo con un for  
29        for i in range(0, self.n):
```

```
print("> Insegno il corso: ", self.lista_corsi[i])
```

A questo punto lo studente è invitato a risolvere il seguente esercizio.

Esercizio 6.5 (Docenti mappati su studenti). Si estendano ulteriormente le classi definite nell'Esempio 31 aggiungendo:

- un algoritmo che calcoli se un docente insegna tutti i corsi frequentati da uno studente.
- un algoritmo che verifichi che, per ogni studente iscritto ad un certo numero di corsi, esistano docenti che effettivamente insegnino quei corsi.

Per risolvere il secondo punto si usi la soluzione del primo.

Per un eccesso di bontà nella scrittura di questa dispensa riportiamo una possibile soluzione. Iniziamo con il definire una funzione ausiliaria - inutile per ora, ma non quando eseguirete il codice! - che stampi un separatore su console.

```
1 def separatore():
2     print("-----")
```

A questo punto vogliamo scrivere una funzione `docente_copre_studente` che stampi a schermo se un docente insegna tutti i corsi frequentati da uno studente. Nell'ottica di risolvere anche il secondo punto dell'esercizio decidiamo di far sì che la funzione restituisca l'eventuale lista di corsi scoperti per lo studente. In notazione insiemistica, se lo studente segue l'insieme dei corsi S , ed il docente insegna l'insieme dei corsi D , allora questa funzione restituisce

$$R = S \setminus D$$

definita mediante una sottrazione tra insiemi. R è l'insieme dei corsi non coperti da questo docente, per questo studente.

```
1 def docente_copre_studente(docente, studente):
2
3     # Convenevoli
4     print("Docente che esamino")
5     separatore()
6     docente.saluta()
7
8     print("\n Studente che esamino")
9     separatore()
10    studente.saluta()
11
12    print("\n Ricerca in corso...")
```

```

13 separatore()
14
15 # flag: True se tutti i corsi sono coperti
16 corsi_trovati = True
17
18 # lista di corsi scoperti, inizialmente vuota
19 # In C questa sarebbe una lista linkata
20 corsi_scoperti = []
21
22 # cerco ogni corso dello studente
23 for i in range(0, studente.n):
24
25     # ipotizzo che il corso non sia stato trovato
26     corso_trovato = False
27
28     # vado a vedere tutti i corsi del docente
29     j = 0
30
31     while not corso_trovato and j < docente.n:
32         if(studente.lista_corsi[i] == docente.lista_corsi[j]):
33             corso_trovato = True
34             j = j + 1
35
36     # stampo l'info che dice se ho trovato o meno il corso
37     print("Corso ", studente.lista_corsi[i], "->",
38           ↪ corso_trovato)
39
40     # se il corso non è stato trovato, appendo in fondo alla
41     ↪ lista
42     if (not corso_trovato):
43         corsi_scoperti = corsi_scoperti +
44         ↪ [studente.lista_corsi[i]]
45
46     # Creo un and tra tutti i corsi
47     corsi_trovati = corsi_trovati and corso_trovato
48
49
50 # riporto a schermo se ho trovato tutto (fuori dal for)
51 print("\n Ricerca completata")
52 separatore()
53
54 if(corsi_trovati):
55     print("Tutti i corsi sono coperti")
56 else:
57     print("Qualche corso rimane scoperto")
58     for i in range(0, len(corsi_scoperti)):
59         print("\t >", corsi_scoperti[i])

```

```
return corsi_scoperti
```

Commentiamo questa funzione:

- come schema di programmazione abbiamo usato la ricerca con flag vista nella Parte I di questa dispensa (Sezione ??); essendo un pattern di programmazione, anche se lo abbiamo visto in `C` lo possiamo usare in `Python`. Anticipiamo però che questo non sia un modo molto pythonico di scrivere questo algoritmo – dopo vedremo come mai!
- nel passaggio

```
corsi_scoperti + [studente.lista_corsi[i]]
```

appendiamo in fondo alla lista `corsi_scoperti` il nome del corso che non troviamo, `studente.lista_corsi[i]` – come ottenevamo con la `addC` nelle liste linkate in `C` (Sezione ??). Qui il `"+"` indica la concatenazione di due liste per la precisione; per poterlo usare possiamo costruire una lista attorno a `studente.lista_corsi[i]`, mediante le parentesi quadrate – una lista di un singolo elemento.

- la formula logica che costruisco è un “and” su ciascuno dei corsi dello studente. La variabile `corso_trovato` mi dice se ho trovato o no il corso *i*-esimo. Quindi `corsi_trovati` – booleano – è inizializzata a `True` altrimenti l’and sarebbe sempre `False`.

Adesso vogliamo scrivere una funzione che verifichi che, per ogni studente iscritto ad un certo numero di corsi, esistano docenti che effettivamente insegnino quei corsi. Ragioniamone da un punto di vista algoritmico, vista la funzione appena definita:

- supponiamo che uno studente debba seguire i corsi $S^{(1)}$ dove il (1) denota il passo di computazione - al primo passo $S^{(1)} = S$ (tutti i corsi sono ancora da coprire) ;
- prendiamo un docente – uno qualunque dei disponibili – che copre i suoi corsi $D^{(1)}$, laddove sperabilmente $D^{(1)} \cap S^{(1)} \neq \emptyset$;
- usiamo la funzione sopra definita per calcolare

$$R^{(1)} = S^{(1)} \setminus D^{(1)}$$

ovvero definire l’insieme dei corsi che non sono coperti dal docente che insegna $D^{(1)}$ (ma magari dagli altri);

- procediamo ricorsivamente – nel senso di definire $S^{(2)} = R^{(1)}$ – a risolvere. A tal proposito possiamo immaginare di risolvere lo stesso problema supponendo di avere a disposizione un nuovo studente con corsi $S^{(2)}$, ed

un pool di docenti uguale a quello iniziale, decurtato del docente che insegna $D^{(1)}$ (già testato).

Chiaramente questo schema di programmazione si basa su una sorta di relazione ricorsiva

$$S^{(i+1)} = S^{(i)} \setminus D^{(i)}$$

che si ferma quando o la lista di docenti da cui prendere un nuovo D è terminata, oppure quando $S^{(i+1)} = \emptyset$. In ambedue i casi, solo se alla terminazione non ho più corsi da coprire allora avremo copertura totale.

Nota bene. Essendo uno studente un oggetto di una classe che noi abbiamo definito, possiamo davvero creare nuovi studenti istanziando la classe **Studente**! Quindi in pratica possiamo davvero creare un nuovo studente per modellare $S^{(i+1)}$.

Possiamo scrivere questa intuizione in Python come segue.

```
1 def copertura_totale(lista_di_docenti, studente):
2
3     # ipotesi iniziale, non è coperto
4     coperto = False
5
6     # inizialmente devo verificare tutti i corsi
7     corsi_da_verificare = studente.lista_corsi
8
9     i = 0
10
11     print("Ricerca copertura")
12     separatore()
13
14     # iterazione indeterminata: appena tutti i corsi sono coperti
15     ↪ esco dall'iterazione
16     while not coperto and i < len(lista_di_docenti):
17
18         # creo un nuovo studente il cui attributo corsi
19         # ha tutti i corsi che non sono stati coperti dai docenti
20         ↪ esaminati fino ad ora
21         obj_S_ast = Studente(studente.nome, studente.cognome,
22         ↪ corsi_da_verificare)
23
24         # calcolo i corsi che non sono coperti, oppure lista vuota
25         copertura_docente =
26         ↪ docente_copre_studente(lista_di_docenti[i], obj_S_ast)
27
28         # se tutti i corsi sono stati coperti
```

```

25     if(len(copertura_docente) == 0):
26         coperto = True
27
28         # se alcuni corsi ancora non sono coperti
29     else:
30         corsi_da_verificare = copertura_docente
31         i = i + 1
32
33     print("Ricerca con tutti i docenti completata")
34     separatore()

```

Si invita lo studente di questa dispensa a testare il codice qui riportato.

Nota bene (Variabili di istanza fuori dal costruttore). In Python, a differenza di altri linguaggi, la struttura di oggetti è piuttosto “flessibile”. Si consideri questo codice.

```

1 class A:
2     def __init__(self, a):
3         self.a = a
4
5     def secondo(self, b):
6         self.b = b

```

Qui `b` è una variabile di istanza degli oggetti di classe `A` a tutti gli effetti, solo che è disponibile solamente dopo aver chiamato il metodo `secondo`. In generale quindi lo stato interno di un oggetto può “aumentare” man mano che certi calcoli (i.e., funzioni) sono applicate agli stessi.

7 Meccanismi avanzati di iterazione

7.1 List comprehensions

In alcuni degli esempi delle Sezioni 6.1 e 6.2 abbiamo esplicitamente criticato lo stile di programmazione adottato, giudicandolo “troppo vicino” a quello visto in C. Infatti, in Python, abbiamo alcuni meccanismi di programmazione che permettono di ottenere lo stesso risultato del codice mostrato precedentemente, con uno sforzo molto minore. Essere un buon programmatore Python richiede quindi di conoscere e sfruttare tali meccanismi.

In questa sezione introduciamo uno di questi meccanismi: la **list comprehension**, un modo semplice per creare liste e sottoliste in Python. La comprehension rende i nostri programmi estremamente più facili da leggere, e rapidi da scrivere. Per introdurla, faremo un confronto con la modalità “classica” di creazione di liste, quella che abbiamo usato negli esercizi precedenti.

Consideriamo che una lista è formata da elementi provenienti da altre sequenze (e.g., i numeri da 1 a 10) su cui sono state svolte determinate operazioni,

o da sotto-sequenze che rispettano determinate condizioni (e.g., i numeri pari da 1 a 10). Ad esempio, supponiamo di aver bisogno di una lista di quadrati a partire da un elenco di numeri.

In modalità **C**, procederemmo come segue:

1. creiamo una lista vuota `lista`;
2. aggiungiamo a `lista` i quadrati dei numeri da elevare, il tutto utilizzando un ciclo `for`;

In Python, usando la `append` – che è esattamente la funzione `addC` della Sezione ?? in **C** – possiamo scrivere

```
1 quadrati = []
2
3 for n in range(10):
4     quadrati.append(n**2)
```

da cui la stampa attesa

```
1 print(quadrati) #[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

La list comprehension per questa lista è invece

```
1 # Una linea
2 quadrati = [n**2 for n in range(10)]
3
4 print(quadrati) #[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

il cui effetto è evidentemente lo stesso – la stessa lista di quadrati viene creata, ma con solo una riga di codice!

La sintassi “astratta” – nella versione più semplice – di una list comprehension segue questo pattern

$$[f(x) \text{ for } x \text{ in } X]$$

così da creare la lista

$$[f(x_1), f(x_2), \dots, f(x_n)]$$

laddove $X = \{x_1, x_2, \dots, x_n\}$. Si noti che è semplice leggere la definizione stessa della lista per capire il significato della comprehension: in questo caso sono tutti e soli gli elementi $x \in X$, ai quali applico f .

Nota bene. Si noti come la comprehension sia formalmente scrivibile con un quantificatore universale ($\forall$) come quelli discussi in Sezione ??

Nell'esempio precedente X sarebbe `range(10)`, ovvero i numeri da 0 a 9, e la funzione f sarebbe il quadrato. Notate che non si definisce una funzione vera e propria, infatti `n**2` è semplicemente una espressione il cui scoping è legato al lato destro della comprehension – quindi a `x in X`.

Esercizio 7.1. Si crei una lista identica a quella di partenza mediante comprehension.

```

1 x = [1,2,3,4,5]
2
3 y = .... # comprehension.
4
5 print(x) # [1,2,3,4,5]
6 print(y) # [1,2,3,4,5] ?

```

Vediamo un altro esempio semplice per complicare un poco la comprehension: vogliamo combinare i valori presenti in due liste, purché questi siano diversi tra loro.

```

1 lista_uno = [1, 2, 3]
2 lista_due = [3, 1, 4]
3
4 mix = []
5
6 # annidamento di cicli for
7 for x in lista_uno:
8     for y in lista_due:
9         if x != y:
10             mix.append((x, y))
11
12 print(mix) # [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3,
↪ 4)]

```

Oppure, in una singola riga con una comprehension

```

1 mix = [(x, y) for x in lista_uno for y in lista_due if x != y]
2
3 print(mix) #[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3,
↪ 4)]

```

Notate che qui usiamo una doppia comprehension, nel senso di quantificare su due insiemi di oggetti. Inoltre, qui utilizziamo anche un predicato per descrivere una appartenenza condizionata – solo se il predicato è vero. La forma generale di comprehension a singola iterazione sarebbe

$$[f(x) \text{ for } x \text{ in } X \text{ if } p(x)] = [f(x_{i_1}), f(x_{i_1}), \dots, f(x_{i_n})]$$

laddove $P_X = \{\mathbf{x}_{i_1}, \mathbf{x}_{i_2}, \dots, \mathbf{x}_{i_n}\} \subseteq X$ è il sottoinsieme

$$P_X = \{\mathbf{x}_i \mid \mathbf{x}_i \in X \wedge p(\mathbf{x}_i)\}.$$

Esercizio 7.2. Si prendano i codici `Python` degli esempi mostrati fino ad ora, in particolare quello riguardante la copertura dei corsi (Esempio 31), e la funzione `def min_value(self, delta_x = 0.5)` nell'esempio riguardante lo studio di funzione (classe `Function`), e li si riscriva mediante list comprehension laddove possibile (e sensato).

7.2 Iteratori

In una comprehension scriviamo

```
[... for x in X]
```

Ora, tutti noi concordiamo nell'intuizione di dire “per ciascun elemento x di X ”, ma un computer che interpreta un programma non può basarsi sull'intuizione. Lo studente attento avrà a questo punto notato che ogni cosa “intuitiva” deve in realtà basarsi su qualcosa di rigoroso, generale e, soprattutto, chiaro.

In questa sezione, introduciamo il concetto di **iteratore**, il modo per navigare (o camminare, o generare, etc.) i valori che sono memorizzati all'interno di una collezione appunto “iterabile”.

Nota bene (Collezioni iterabili). Una collezione iterabile è un modo di rappresentare gli oggetti secondo una qualche struttura, definendo il concetto di cammino sugli stessi. Per esempio, se pensiamo ad i numeri da 1 a 10, il cammino più semplice è quello che genera

$$1, 2, 3, \dots, 10$$

In alternativa, se penso ad una stringa “**banana**”, il cammino più semplice potrebbe essere quello che scandisce le lettere di “**banana**” una per ciascuna

$$\text{"b", "a", "n", \dots, "a"}$$

Adesso, si noti bene che il modo in cui si decide di fare un determinato cammino è completamente arbitrario. Potrei, per motivi miei, decidere di saltare a caso fra i numeri da 1 a 10, oppure saltare solo tra da un pari all'altro! L'importante, è che se una collezione è iterabile, allora posso generare dei cammini su di essa.

In `Python` un iteratore è un oggetto su cui è possibile iterare, ovvero un oggetto che ha una nozione *implicita* di iterazione. Il meccanismo di iterazione ritorna un elemento per volta e viene realizzato implementando necessariamente due magic methods:

- `__iter__`, che inizializza l'iteratore e lo ritorna tramite `self` (perchè l'iteratore essendo implicito fa parte dell'oggetto stesso, quindi è `self`);
- `__next__`, che ritorna il prossimo valore dell'iteratore da visitare.

Cominciamo con vedere il semplice utilizzo di un iteratore

```

1 mystr = "banana"
2 myiter = iter(mystr) # costruisco iteratore
3
4 print(next(myiter)) # b
5 print(next(myiter)) # a
6 print(next(myiter)) # n
7 print(next(myiter)) # a
8 print(next(myiter)) # n
9 print(next(myiter)) # a

```

In questo caso costruisco l'iteratore con la chiamata `iter(mystr)`, usando il fatto che qualcuno ha implementato un magic method `__iter__` per le stringhe in Python. Le chiamate che restituiscono i valori su cui si muove l'iteratore - in questo caso i caratteri della stringa - sono tutte della forma `next(myiter)`.

Esempio 32 (Iteratore indefinito). Costruiamo un semplicissimo iteratore che generi i numeri in modo indefinito, i.e.,

$$1, 2, 3, 4, \dots$$

Per farlo, l'iteratore l'unica cosa che deve ricordarsi è l'ultimo valore, chiamiamolo a , che ha restituito nelle chiamate a `next`. Per ottenere questo effetto di “memoria” basta utilizzare le variabili di istanza dell'oggetto che stiamo costruendo – aggiungiamo una variabile `a` a `self`!

```

1 class MyNumbers:
2
3     # Costruzione dell'iteratore 1, 2, ...
4     def __iter__(self):
5         self.a = 1 # self.a viene inizializzato ad 1
6         # iter deve ritornare l'oggetto stesso
7         return self
8
9     def __next__(self):
10        x = self.a # copia di self.a in x
11        self.a += 1 # incremento self.a di 1
12        return x # restituisco x

```

Avendo una concezione ad oggetti per gli iteratori, posso facilmente testare la nostra implementazione istanziando un oggetto di classe `MyNumbers`.

```
1 myclass = MyNumbers()
2 myiter = iter(myclass) # costruisco iteratore
3
4 print(next(myiter)) # 1
5 print(next(myiter)) # 2
6 print(next(myiter)) # 3
7 print(next(myiter)) # 4
```

Ad ogni modo, `mystr` ha un numero finito di caratteri, che cosa succede se chiamo un'altra volta `next(myiter)`?

```
1 print(next(i)) # Eccezione: StopIteration!
```

Viene sollevata quindi una eccezione di tipo `StopIteration`; si veda la dispensa di Laboratorio di Programmazione per un'introduzione alla gestione delle eccezioni in Python. Per fermare una iterazione dobbiamo sollevare una eccezione.

Esempio 33 (Iterazione definita). Prendiamo l'esempio precedente, ora vogliamo però definire un iteratore che generi tutti i valori interi dell'intervallo $[a, b]$, perciò

$$a, a + 1, a + 2, \dots, b$$

Ovviamente, gli estremi a e b dell'intervallo possono essere passati a tempo di creazione dell'oggetto su cui costruirò l'iteratore mediante due parametri `low` e `high`.

```
1 class Series:
2
3     # Costruttore, definisce a e b
4     def __init__(self, low, high):
5         # variabile locale che dice a quale punto sono della
6         #   ↳ serie
7         self.current = low
8         self.high = high # costante
9
10    # Se ho già settato current a low, posso non fare nulla
11    def __iter__(self):
12        return self
13
14    def __next__(self):
```

```

14         # se sono fuori dell'intervallo di riferimento
15         if self.current > self.high:
16             # sollevo un'eccezione
17             raise StopIteration
18         else:
19             self.current += 1
20             return self.current - 1

```

La cosa confortevole di un iteratore è che, dopo averlo costruito, posso alleggerire la scrittura del mio codice sfruttandolo. Per esempio, riferendoci all'Esempio 33 in questo caso possiamo ciclare esplicitamente sugli elementi della nostra serie usando un ciclo `for`.

```

1  # 1, 2, 3, ... , 10
2  myS = Series(1, 10)
3
4  # ciclo esplicitamente sulla classe
5  for s in myS:
6      print(s)

```

In maniera analoga, possiamo usare una list comprehension al posto del `for`.

```

1  [print(s) for s in Series(1, 10)]

```

E, senza stupirci, possiamo costruire una lista Python a partire dai nostri valori iterabili.

```

1  # costruisco una lista a partire da un oggetto di tipo Series
2  n_list = Series(1, 10)
3  print(list(n_list))

```

Nota bene (Interruzione di iterazione). Si osservi come sollevare esplicitamente l'eccezione `StopIteration` serva a interrompere l'iterazione, che altrimenti non si fermerebbe mai!

Il pattern generale per scorrere il nostro iteratore è quindi:

```

1  # inizio l'iterazione
2  for element in iterable:
3      # faccio qualcosa con element

```

Questo pattern corrisponde in realtà al seguente codice.

```

1  # creo iteratore da oggetto iterabile
2  iter_obj = iter(iterable)
3
4  # loop infinito

```

```

5 while True:
6     try:
7         # prendi il successivo elemento
8         element = next(iter_obj)
9
10        # faccio qualcosa con element
11
12    # in caso di StopIteration, esci dal loop
13    except StopIteration:
14        break

```

Esercizio 7.3. Come mai la “semantica” dell’iteratore viene implementata con un costrutto `while`?

Possiamo utilizzare il concetto di iteratore per definire un flusso di dati che magari non sono stati pre-calcolati. Ad esempio, le potenze di un certo numero.

Esempio 34. Vogliamo creare un iteratore che stampi tutte le potenze di 2, fino a che non raggiunge un esponente massimo x , ovvero

$$2^0, 2^1, 2^2, \dots 2^x$$

```

1 class PowTwo:
2     # ricordo x
3     def __init__(self, x):
4         self.max = x
5
6     # mi preparo ad iterare da 2^0=1
7     def __iter__(self):
8         # esponente corrente
9         self.n = 0 # inizio da 2^0
10        return self
11
12    def __next__(self):
13        if self.n <= self.max:
14            result = 2**self.n
15            self.n += 1
16            return result
17        else:
18            raise StopIteration

```

A questo punto la class `PowTwo` può essere utilizzata per generare il flusso dei dati previsti.

```

1 numbers = PowTwo(3)
2 i = iter(numbers)
3
4 print(next(i)) # 2^0 = 1
5 print(next(i)) # 2^1 = 2
6 print(next(i)) # 2^2 = 4
7 print(next(i)) # 2^3 = 8
8 print(next(i)) # Eccezione: StopIteration

```

Esempio 35 (Successione di Fibonacci). Si vuole costruire un iteratore che generi i termini F_i della successione di Fibonacci, limitandolo a calcolare F_M con M un massimo scelto dall'utente.

```

1 class MyFibo:
2
3     def __init__(self, max):
4         # massimo indice dei termini generati
5         self.max = max
6
7     def __iter__(self):
8         self.n = 0 # elemento indice 0 della successione
9         self.n_0 = 0 # F_0
10        self.n_1 = 1 # F_1
11        return self
12
13    def __next__(self):
14        # controllo se sono all'interno dei valori che devo
15        # → calcolare
16        if self.n <= self.max:
17            if self.n == 0:
18                # ho memorizzato il valore
19                result = self.n_0
20            if self.n == 1:
21                # ho memorizzato il valore
22                result = self.n_1
23            if self.n > 1:
24                # termine n-esimo F_n
25                result = self.n_0 + self.n_1
26                # memorizzo F_{n-1}
27                self.n_0 = self.n_1
28                # memorizzo F_n
29                self.n_1 = result
30            # incremento n
31            self.n += 1
32            return result
33        else:

```

```

33         raise StopIteration
34
35     # esempio di utilizzo
36     fibo = MyFibo(20)
37     for i in fibo:
38         print(i)

```

Riprendendo le classi viste negli esempi precedenti riguardanti studenti e docenti (Sezione 6.1), supponiamo di voler modellare una università intesa come collezione di studenti e docenti. In particolare, vogliamo implementare un iteratore nella classe `Università` che permetta di eseguire qualche operazione per ogni `Studente` iscritto all'università.

```

1  class Università:
2
3      # Memorizzo le liste nel costruttore
4      def __init__(self, lista_studenti, lista_docenti):
5          self.lista_studenti = lista_studenti
6          self.lista_docenti = lista_docenti
7
8      # Inizialmente non ho ancora visto nulla
9      def __iter__(self):
10         self.indice = -1
11         return self
12
13     def __next__(self):
14         # prima incremento indice dato che parto da -1
15         self.indice += 1
16         if(self.indice == len(self.lista_studenti)):
17             raise StopIteration
18         return self.lista_studenti[self.indice]

```

Grazie a questo semplice iteratore è possibile scrivere codice estremamente leggibile e versatile, sfruttando ad esempio una list comprehension.

```

1  # Creo una università
2  universita_units = Università([obj_Giulio, obj_Mario],
3      ↪ [obj_ProfRusso, obj_ProfPippo])
4
5  # Faccio qualcosa per ciascuno studente
6  [studente.bonjour() for studente in universita_units]

```

7.3 Liste concatenate in Python

Concludiamo questa dispensa costruendo, in Python, una struttura dati analoga alla lista linkata che abbiamo implementato in C.

```

1 class ListaConcatenata:
2     # inner class: visibile solo all'interno della classe
3     ↪ ListaConcatenata
4     # analogo a ElementoDiLista in C
5     class Nodo:
6         val = None # valore interno al nodo
7         prossimo = None # elemento next della struct in C
8
9     # costruttore
10    # stessa funzione del puntatore ListaDiElementi in C
11    def __init__(self):
12        self.testa = None # puntatore alla testa della lista
13
14    # funzione che aggiunge un elemento alla lista
15    # stessa funzionalità di addC in C
16    def aggiungi(self, n_val):
17        # se la lista è vuota
18        if self.testa == None:
19            # costruisco un nodo
20            n_nodo = self.Nodo()
21            # gli assegno il valore in input
22            n_nodo.val = n_val
23            # assegno il valore di testa
24            self.testa = n_nodo
25        # se la lista contiene già almeno un elemento
26        else:
27            # copio la testa, poiché non voglio modificare
28            ↪ self.testa
29            elem = self.testa
30            # quando elem.prossimo == None sono all'ultimo elemento
31            ↪ della lista
32            while elem.prossimo != None:
33                # sposto elem a elem.prossimo
34                elem = elem.prossimo
35                # creo un nuovo nodo
36                n_nodo = self.Nodo()
37                n_nodo.val = n_val
38                elem.prossimo = n_nodo
39
40    # esempio di utilizzo

```

Nota bene. Dal momento che Python non è un linguaggio fortemente tipato, il campo `val` può contenere oggetti di qualunque tipo.

E' naturale pensare di implementare un iteratore sulle liste, ovvero posso aggiungere alla classe definita le seguenti funzioni:


```

1  # ... all'interno della classe ListaConcatenata ...
2  def __iter__(self):
3      # itero a partire dall'inizio della lista
4      self.corrente = self.testa
5      return self
6
7  def __next__(self):
8      # se ho raggiunto la fine della lista
9      if self.corrente is None:
10         raise StopIteration
11     # se ci sono ancora elementi da visitare
12     val_corrente = self.corrente.val # valore oggetto corrente
13     # mi sposto al prossimo elemento della lista
14     self.corrente = self.corrente.prossimo
15     return val_corrente

```

Con queste liste fatte in casa possiamo fare le seguenti operazioni.

```

1  lista_con = ListaConcatenata()
2
3  # Aggiungo elementi
4  lista_con.aggiungi(3)
5  lista_con.aggiungi(4)
6
7  # Posso usare gli stessi pattern di programmazione visti sulle
   ↳ liste in C
8  # ad esempio per stampare tutti gli elementi della lista:
9  corrente = lista_con.testa
10 while corrente is not None:
11     print(corrente.val)
12     corrente = corrente.prossimo
13
14 # Oppure posso sfruttare implicitamente l'iteratore creato
15 for elem in lista_con:
16     print(elem)

```

Gestione degli errori e dell'input

8 Le eccezioni

In Python, una qualsiasi interruzione del normale flusso del codice è chiamata *eccezione*. Queste interruzioni sono quasi sempre errori, ma non è sempre così. Per esempio, internamente a Python, per segnalare la fine di un ciclo si usa una eccezione che rappresenta la fine del ciclo. Detto questo, possiamo in prima approssimazione pensare alle eccezioni come agli errori, perché è in effetti quasi sempre così.

Le eccezioni, come tutto in Python, sono oggetti, e in particolare estendono sempre la classe base. *Exception*. Vediamone alcuni esempi:

- **Exception:**
 - **ArithmeticError**: problemi nel calcolo numerico;
 - * **FloatingPointError**: problemi nel calcolo numerico
 - * **ZeroDivisionError**: divisione per zero
 - **AttributeError**: l'attributo dell'oggetto non esiste
 - **NameError**: variabile non definita
 - **SyntaxError**: il codice non è semanticamente corretto
 - **TypeError**: tipo dati inappropriato
 - **ValueError**: valore inappropriato

Quando si verifica un'eccezione, il comportamento di default è di interrompere l'esecuzione del codice dove è avvenuta, e di stamparla a schermo assieme al suo *traceback*. Questo è un “report” automaticamente generato da Python che serve ad identificare dove è avvenuta l'eccezione. Quello che fa è proprio di *rintracciare all'indietro* le chiamate delle funzioni che hanno portato al suo verificarsi.

```
> python lezione5.py
Traceback (most recent call last):
  File "lezione5.py", line 3, in <module>
    float(my_var)
ValueError: could not convert string to float: 'Ciao'
```

Figura 6. Una eccezione con il suo traceback.

Errori recoverable e non Un errore *recoverable* non causa un crash del programma ma può portare ad una perdita di dati. Nel caso di input non corretto, infatti, si hanno due opzioni:

- stampare l'errore, usare un valore di default e continuare con l'esecuzione del programma (errore recoverable);
- stampare l'errore o un'eccezione personalizzata e uscire dal programma (errore non recoverable).

Riprendendo l'esempio della stampa delle prime *n* righe di un file, si può pensare che un utente passi come valore 3.5. A tutti gli effetti questo è un errore, in quanto ci si aspettava un int, ma lo si può *sanitizzare* usando il casting. Invece, nel caso in cui venisse passata la stringa 'a', allora l'errore sarebbe non recoverable se non si disponesse di un valore di default da sostituire e il programma non potrebbe continuare.

Si noti che l'uscita dal programma si effettua solo in caso di input proveniente dall'esterno, in quanto sarebbe sintomo di design errato uscire da un programma i cui valori vengono passati dal programma stesso.

Il costrutto try-except Il costrutto *try-except* serve a gestire le eccezioni che possono insorgere all'interno del suo corpo. Si può così reagire ad un errore, in vari modi. In alcuni casi si riesce a “recuperare”: se si deve leggere un file con dei parametri ottimizzati per una simulazione, si può probabilmente comunque proseguire con i parametri di default. In altri casi, si può segnalare l'errore ma proseguire con l'esecuzione del programma senza farlo terminare brutalmente: pensiamo per esempio alle interfacce grafiche o agli applicativi Web. In altri casi ancora, non si può fare niente e allora si lascia che l'eccezione “salga” a Python, senza gestirla, che interromperà l'esecuzione del programma e la stamperà a schermo con il suo traceback, come nell'immagine precedente.

Più in dettaglio, il costrutto **try-except** prova ad eseguire il blocco di istruzioni indentato che segue la parola chiave **try**, e:

- se non si presentano eccezioni, salta alla prima istruzione successiva al costrutto stesso (cioè dopo il blocco di codice dell' **except**);
- se invece si presenta un'eccezione, il costrutto esegue il blocco di codice dell' **except**.

Esempio 36. Esempio di utilizzo del costrutto try-except.

```
1 mia_var = 'ciao'
2 try:
3     mia_var = float(mia_var)
4 except:
5     print('Non posso convertire "mia_var" a numero!')
```

```

6     print('Uso il valore di default di "0.0"')
7     mia_var = 0.0
8     print(mia_var) # Stampa a schermo 0.0

```

Nell'esempio appena visto l'eccezione `ValueError`, che si presenta quando si prova a convertire una stringa alfabetica a float, viene catturata e gestita dall' `except`. Si noti che non è stato specificato il tipo di eccezione da gestire, quindi Python è pronto a gestirne una qualsiasi. Possiamo però gestire solo specifiche eccezioni, specificandone il tipo nell' `except`.

In questo modo il costrutto `try-except` passerà in rassegna tutti i tipi di eccezioni specificate dalle varie `except`, e verificherà se corrisponde ad una di esse (la prima che trova):

- se corrisponde, eseguirà il relativo blocco di codice, senza bloccare l'esecuzione del programma;
- se invece non corrisponde a nessuna di esse, Python bloccherà l'esecuzione del programma e riporterà l'eccezione a schermo con il suo traceback, come se non ci fosse stato nessun costrutto `try-except`.

Si può inoltre “nominare” un'eccezione gestita in un `except` usando la parola chiave `as`: questo la rende disponibile nel blocco, ed essendo un'eccezione un oggetto come un altro, può essere manipolata, stampata a schermo o altro, come ad esempio essere trascritta su un file di log.

Esempio 37. Nel seguente snippet, si chiede a Python di gestire l'errore in modo diverso a seconda dell'eccezione, e di gestire un'eccezione non coperta dai casi previsti stampandola a schermo.

```

1     try:
2         mia_var = float(mia_var)
3     except ValueError:
4         print('Non posso convertire "mia_var" a valore numerico!')
5         print('Errore di VALORE, "mia_var" valeva
6             ↳ "{}".format(mia_var))
7     except TypeError:
8         print('Non posso convertire "mia_var" a valore numerico!')
9         print('Errore di TIPO, "mia_var" era
10            ↳ "{}".format(type(mia_var)))
11     except Exception as error:
12         print('Non posso convertire "mia_var" a valore numerico!')
13         print('Errore generico: "{}".format(error))

```

L'ultima clausola, effettuando il controllo sulla classe base `Exception`, andrà a gestire tutte le eccezioni, al pari dell' `except` semplice, ma l'utilizzato

di questo costrutto ci ha permesso di dare un nome all'eccezione (variabile `error`) per poi stamparla a schermo.

Si possono usare anche due ulteriori funzionalità del costrutto `try-except`, che sono rispettivamente le parole chiave *else* e *finally*. Il blocco di codice che segue l' `else` viene eseguito nel caso non ci sia stata nessuna eccezione, mentre quello che segue il `finally` viene eseguito sempre, anche in caso di errore, prima di interrompere l'esecuzione del programma. Questo è molto utile per “far pulizia”, come ad esempio chiudere i file aperti. In generale, il costrutto nella sua interezza è da intendersi così:

- Prova (`try`):
 - fai questa cosa
- Se c'è questo tipo di errore (primo `except`):
 - fai quest'altra cosa
- Se c'è quest'altro tipo di errore (secondo `except`):
 - fai quest'altra cosa ancora
- Altrimenti, se non ci sono errori (`else`):
 - fai questo
- E in ogni caso (`finally`):
 - fai quest'altro

Un esempio pratico potrebbe per esempio essere la lettura di un parametro da un file, come esposto nell'esempio sottostante.

Esempio 38. Lettura di un parametro da file con gestione della maggior parte delle casistiche di errore.

```
1 parametro = 0
2 nome_file_parametro = 'parametro.txt'
3 try:
4     file_parametro = open(nome_file_parametro)
5     parametro_come_stringa = file_parametro.read()
6     parametro_come_float = float(parametro_come_stringa)
7 except IOError:
8     print('Non posso leggere il file!')
9     print('Userò il valore di default per il parametro')
10 except ValueError:
11     print('Non posso convertire il parametro a valore
12         ↳ numerico!')
13     print('Errore di valore, il parametro valeva
14         ↳ "{}".format(parametro_come_stringa))
```

```

13     print('Userò il valore di default per il parametro')
14 except Exception as error:
15     print('Errore generico: "{}".format(error))
16     print('Userò il valore di default per il parametro')
17 else:
18     parametro = parametro_come_float
19 finally:
20     file_parametro.close()

```

Tramite l' `else`, la variabile `parametro` viene aggiornata con il valore corretto solo se non si verificano problemi nell'apertura del file o nella conversione del valore. Questo garantisce che il valore venga impostato solo quando il flusso è integro. Tramite il `finally`, assicuro che il file aperto con `open()` venga sempre chiuso.

Da notare che questo costrutto `try-except` gestisce eccezioni multiple e diverse anche nella loro natura, poiché potrebbero insorgere sia dall'apertura del file che dalla conversione del parametro a float. Si sarebbero potuti scrivere due costrutti `try-except` distinti nel caso fosse servito essere più granulari.

Da notare anche che si sarebbero potuti *annidare* i costrutti: per prima cosa si poteva provare ad aprire il file e poi si poteva scrivere un ulteriore `try-except` annidato, da eseguire solo nel caso in cui l'apertura del file fosse avvenuta senza errori, per provare a convertire il parametro in float.

Generare eccezioni Per generare un'eccezione si usa la parola chiave *raise*, seguita da una istanza dell'eccezione. Si potrebbe voler generare (o, *alzare*) un'eccezione per svariati motivi, come ad esempio quando un valore di input non è tra quelli ammessi, o perché si sono verificate delle condizioni particolari che non sono gestite nel codice.

L'argomento da passare all'eccezione quando viene istanziata è il “messaggio che porta con sé”, ovvero quello che verrà stampato a schermo nel caso in cui nel codice questa eccezione non venga gestita da nessun costrutto `try-except`.

Esempio 39. Generazione di un'eccezione.

```

1 parametro = -5
2 if parametro < 0:
3     raise ValueError('Il parametro è minore di zero')

```

Si potrebbe anche voler definire un'eccezione personalizzata. In tal caso basterebbe estendere la classe base `Exception` con un corpo vuoto (cioè usando l'istruzione nulla, che in Python si indica con *pass*).

Esempio 40. Definizione e generazione di un'eccezione personalizzata.

```
1 class InvalidParameter(Exception):  
2     pass  
3  
4 parametro = -5  
5 if parametro < 0:  
6     raise InvalidParameter('Il parametro è minore di zero')
```

9 Controllo degli input

Un *input* è un qualsiasi dato che “entra da qualche parte”. Una funzione riceve degli argomenti in input, la lettura di un file porta dei dati come input al programma e anche la scrittura dell'utente su linea di comando è un tipo di input. Un input può essere sia già presente nel codice che passato dall'esterno.

Non fidarsi degli input Non ci si deve mai fidare della bontà degli input, perché non si ha il controllo su chi o cosa li genera. Se i dati sono in formato sbagliato, danneggiati, fuori range o in generale non adatti per l'uso che se ne deve fare, il programma può andare in errore oppure, peggio, non mostrare nessun segno di problema ma dare risultati sbagliati. A seconda dei casi può quindi essere desiderabile controllare che gli input rispettino certe *ipotesi di lavoro* prefissate.

Ad esempio, si supponga di creare una funzione che legga le prime *n* righe di un file, dove *n* è passato come input dall'utente. Si dovrebbe controllare che:

- *n* sia un numero;
- *n* sia positivo;
- *n* sia minore o uguale al numero delle righe del file;
- *n* sia intero (si potrebbe accettare anche un `float` ma servirebbe decidere a priori come interpretarlo).

Ogni situazione necessita di ipotesi di lavoro adeguate, da definire in base al contesto, ed è fondamentale prendere in considerazione anche *edge cases*², ovvero non solo i casi ideali in cui tutto va come sperato, ma prepararsi a tutte le eventuali condizioni rare che possono verificarsi, perché è su quelle che in genere sorgono la maggior parte dei problemi. Questo concetto è vero in generale e non solo nella gestione degli input, ma essendo gli input una sorgente enorme di incertezza ci focalizziamo su questa.

²Di norma si cita l'esempio delle [mucche sferiche](#).

Prevenire o correggere? L'approccio alla gestione degli input e degli errori (cioè, di fatto, la maggior parte della logica scritta nei programmi) può seguire due grandi filosofie: prevenire o correggere.

In genere, i linguaggi tipizzati (come C e Java) preferiscono prevenire tutto: viene richiesto che l'input sia esattamente di quel tipo, oppure il codice non può proprio funzionare. In Python, che non è un linguaggio tipizzato, è invece in genere preferita la logica del "prima provo, poi chiedo scusa". Ovvero non si cerca sempre di prevenire, ma piuttosto di correggere gli errori (cioè, gestendo le eccezioni) laddove si verificano.

Tuttavia, anche in Python può spesso avere molto senso controllare prima alcune cose. Se infatti controllare a priori il tipo dati ricevuto in input per verificare che sia corretto può in generale portare a posizioni diverse su quale sia il miglior approccio da utilizzare e quali siano i diversi controlli da fare, ci sono invece casi in cui l'approccio è forzato. Ad esempio, se un programma è pensato per lavorare con un parametro entro un determinato range, allora va sempre verificato che questa condizione sia soddisfatta. Un caso di questo tipo si verifica ad esempio in presenza di un parametro numerico di cui poi si va a fare il logaritmo. In questo caso, il valore del parametro deve per forza essere positivo. Un altro esempio è quello di una stringa che identifica il sesso biologico di una persona, che può esser solo 'M' o 'F'. E' vero che anche in questo caso potrei non controllare niente e "provare" per poi eventualmente "chiedere scusa", ma è anche vero che la gestione diventa molto più complicata e diventa sempre più difficile riuscire ad identificare la causa dell'errore. Scrivere buon codice in Python significa trovare il giusto equilibrio tra controllare a priori l'aderenza dei dati in input alle ipotesi di lavoro e non controllare niente, reagendo laddove si generi un eventuale errore.

Validità degli input Per testare il tipo degli input si usano alcune funzioni built-in che lavorano sugli oggetti, come ad esempio `type`, `isinstance` o `issubclass`.

Esempio 41. Controllo a priori del tipo di input di una funzione che somma tutti gli elementi di una lista.

```
1 def somma_lista(lista_input):
2     if not type(lista_input) == list:
3         raise TypeError('Il tipo di lista_input non è lista ' +
4             'ma "{}".format(type(lista_input)))
5     somma = 0
6     for elemento in lista_input:
7         somma += elemento
8     return somma
```

Nell'esempio sopra in realtà controllare a priori il tipo dati potrebbe non essere una buona idea, perchè la funzione `somma_lista()` potrebbe tranquillamente funzionare anche su una tupla, o una qualsiasi altra struttura che ne permette l'iterazione. Si potrebbe rinominare la funzione in qualcosa tipo

`somma_tutti()` e togliere il controllo, ammettendo anche le tuple, i set, e magari anche altro, lasciando all'utilizzatore il compito di passare in input una struttura dati appropriata. Tuttavia, possiamo sempre decidere come in questo caso (in modo del tutto arbitrario) di volere una funzione che sommi solo ed esclusivamente gli elementi di strutture di tipo lista. Si potrebbe però rilassare un po' il controllo, accettando non solo il tipo dati lista ma anche le sue estensioni (classi ottenute estendendo la classe `lista`). Ad esempio, uno potrebbe voler estendere la classe `lista` con l'obiettivo di aggiungere determinate funzionalità, come nell'esempio sottostante:

Esempio 42. Definisco una nuova classe `DataSet` che estende la classe `lista` aggiungendo una funzione che ritorna il valore assoluto della media degli elementi della lista.

```
1 class DataSet(lista):
2     def get_avg_abs_value(self):
3         return sum([abs(element) for element in
           ↪ self])/len(self)
```

A questo punto, il check effettuato con il `type` non passerebbe più, poiché la classe `DataSet` non è di tipo lista. E' una classe diversa, che però la estende, quindi mantiene tutte le funzionalità della lista e ne aggiunge altre (il metodo `get_avg_abs_value`, in questo caso). Per ovviare a questo inconveniente, è in generale preferibile usare `isinstance` nei controlli di tipo, a meno che non si voglia fissare intenzionalmente il tipo dati a solo una classe specifica. L'esempio sottostante mostra un possibile utilizzo della funzione `isinstance`:

Esempio 43. Controllo a priori del tipo di input di una funzione che somma tutti gli elementi di una lista accettando anche le sottoclassi del tipo dati lista.

```
1 def somma_lista(lista_input):
2     if not isinstance(lista_input, lista):
3         raise TypeError('Il tipo di lista_input non è lista, '
           ↪ +
4                             'ma "{}".format(type(lista_input)))
5     somma = 0
6     for elemento in lista_input:
7         somma += elemento
8     return somma
```

In alcuni casi potrei voler controllare anche il tipo dei singoli elementi della lista, come nell'esempio sottostante:

Esempio 44. Controllo a priori del tipo di input di una funzione che somma tutti gli elementi di una lista, controllando che i suoi elementi siano di tipo intero o float.

```
1 def somma_lista(lista_input):
2     if not isinstance(lista_input, list):
3         raise TypeError('Il tipo di lista_input non è lista', +
4                         'ma "{}"'.format(type(lista_input)))
5     for i, element in enumerate(lista_input):
6         if not (isinstance(element, int) or isinstance(element,
7     ↪ float)):
8         raise TypeError('Il tipo di elemento in posizione
9     ↪ {} di '.format(i) +
10                        'lista_input non è nè intero nè
11                        ↪ float, ma ' +
12                        '"{}"'.format(type(element)))
13
14     somma = 0
15     for elemento in lista_input:
16         somma += elemento
17     return somma
```

Tuttavia, come precedentemente accennato, decidere se effettuare o meno questo tipo di controlli dipende dal caso specifico, da quanto si vuole essere “rigidi” e da come si prevede di utilizzare il codice, perché introducono, tra l’altro, dei rallentamenti nell’esecuzione.

Si noti inoltre come in tutti gli esempi precedenti quando si genera un’eccezione si riporta nel messaggio di errore anche il tipo avuto come non valido: è fondamentale dare sempre più contesto possibile nella gestione degli errori derivanti da input non ammessi, per aiutarci nel debugging evitando messaggi di errore sterili del tipo “Errore”, che non ci danno alcun indizio sul cosa possa essere andato storto.

Sanitizzazione degli input In alcuni casi ci si trova di fronte ad input che risulterebbero in prima battuta non validi ma che potrebbero portare comunque l’informazione necessaria per l’esecuzione del codice. In questo caso, è possibile provare a pulirli, o meglio, a *sanitizzarli*. Consideriamo ad esempio il caso accennato in precedenza in cui si richiede in input il sesso biologico di una persona. Un controllo stretto dell’input potrebbe essere implementato come nell’esempio che segue:

Esempio 45. Nel seguente esempio si vuole controllare se la stringa contenuta nella variabile `sesso` rappresenta uno dei due sessi biologici validi ('M' o 'F').

```
1 if sesso not in ['M', 'F']:
```

```

2     raise ValueError('Il valore del sesso non è nè M ne F, ' +
3                       'ma invece vale "{}"'.format(sesso))

```

Notiamo però che questo controllo è molto restrittivo. Infatti, basterebbe anche solo presentare come input il valore ' m' e il controllo non passerebbe più, poiché tale stringa contiene uno spazio ed è in minuscolo. Con alcune semplici considerazioni si può però considerare tale input come comunque valido:

Esempio 46. Nel seguente esempio si vuole controllare se la stringa contenuta nella variabile `sesso` rappresenta uno dei due sessi biologici validi ('M' o 'F'), sanitizzandola prima di effettuare il controllo.

```

1
2     # Si trasforma l'input in maiuscolo
3     sesso = sesso.upper()
4
5     # Si rimuovono gli spazi
6     sesso = sesso.strip()
7
8     # Ora il controllo passa anche per stringhe del tipo ' m'
9     if sesso not in ['M', 'F']:
10         raise ValueError('Il valore del sesso non è nè M ne F, ' +
11                           'ma invece vale "{}"'.format(sesso))

```

In generale, ci si deve sempre chiedere cosa può andare storto quando si utilizza un input, e decidere se non controllarlo per niente (che è una scelta valida, se intenzionale e giustificata) oppure se controllare che sia valido e in che misura. Va in ogni caso fatto uno studio a priori dei possibili problemi, in modo da aver sempre chiaro cosa può, per l'appunto, andare storto.

Esercizi

PART

V

Questa parte è ancora molto parziale. Ci sono solo alcuni esercizi rispetto a tutti quelli visti in classe.

10 Ereditarietà

Esercizio 10.1. Data la classe

```
1 class Veicolo:
2
3     def __init__(self, modello, velocita, km):
4         self.modello = modello
5         self.velocita = velocita
6         self.km = km
```

Si crei una classe **Autobus** che contenga, oltre alle informazioni su un veicolo, la capienza massima e le informazioni sulla rotta seguita (tramite una lista). Si forniscano dei metodi di stampa - nella super-classe o nella sotto-classe? - che stampano informazione su un veicolo generico, e su un autobus.

Esercizio 10.2. Con riferimento all'esercizio precedente, si aggiunga la classe **Auto** (oltre ad **Autobus**) con un metodo di istanza **revisionata** per controllare se la revisione sia stata fatta, o meno. Per verificare ciò **revisionata** prende in input l'anno attuale (**anno**), e testa se il veicolo è coperto per l'anno in input. A tal proposito, si devono chiaramente aggiungere eventuali informazioni riguardanti la revisione tramite variabili interne; si stabilisca vantaggi e svantaggi di metterli nella super-classe o nella sotto-classe, e si implementi l'approccio ritenuto migliore.

Esercizio 10.3. Con riferimento all'esercizio precedente, vogliamo fare le revisioni anche agli oggetti di classe **Autobus**. Si discuta se, con la soluzione svolta per l'esercizio precedente, l'aggiunta ora richiesta richiede tanto o poco aggiornamento di codice. Si risolva nuovamente questo ed il precedente esercizio per ridurre al minimo il codice necessario per avere le revisioni disponibili negli autobus.

Esercizio 10.4. Si crei una sotto-classe **Autobus** della super-class **Veicolo**. Il costo di default per una corsa completa su di un veicolo è 100 volte la capacità (in posti) del veicolo. Se il veicolo è un bus, c'è una maggiorazione del 10% sul costo di base (e.g., se la capacità del bus è di 50 posti, il costo dovrà essere 5500). Si usi il seguente codice.

```

1 class Veicolo:
2     def __init__(self, modello, km, posti):
3         self.modello = modello
4         self.km = km
5         self.posti = posti
6
7     def fare(self):
8         return self.posti * 100
9
10 class Bus(Veicolo):
11     pass
12
13 scuolabus = Bus("Volvo", 12, 50)
14 print("Totale Bus:", scuolabus.fare())

```

Si noti che per implementare il calcolo si richiede l'overriding del metodo `fare()`.

Esercizio 10.5 (Esponenti e potenza). Si scriva una classe potenza per definire le potenze di numeri con esponente sia positivo che negativo.

Esercizio 10.6 (Insieme potenza). Si consideri il seguente codice

```

1 class pset:
2     def sub_sets(self, sset):
3         return self.subsetsRecur([], sorted(sset))
4
5     def subsetsRecur(self, current, sset):
6         if sset:
7             return self.subsetsRecur(current, sset[1:]) +
8                 ↪ self.subsetsRecur(current + [sset[0]], sset[1:])
9         return [current]

```

Si spieghi che tipo di calcolo viene effettuato da questa classe.

Nota bene. Le slice di liste sono introdotte nella dispensa del corso di Laboratorio di Programmazione!

Esercizio 10.7 (Compagnia di viaggi). La compagnia di viaggio “PT - Partire per non Tornare” ha ingaggiato il vostro team per realizzare un programma in grado di gestire i viaggi ed i clienti: i viaggi hanno molte caratteristiche in comune e qualche particolarità che le contraddistingue in base alla tipologia di viaggio (Mare, Montagna, Invernale o Estiva).

Attributi comuni:

- nome viaggio, identifica il titolo (a tema) di una vacanza;
- data partenza, identifica la data di inizio della vacanza e può essere rappresentato come una lista di tre elementi (giorno, mese, anno);

- data ritorno, identifica la data di conclusione della vacanza, anche essa può essere rappresentata come una lista di tre elementi;
- località, identifica la destinazione per la vacanza;
- resort, identifica l'albergo o il complesso turistico dove alloggeranno i clienti;
- prezzo, identifica il prezzo, a persona, per poter partecipare alla vacanza;
- partecipanti, identifica l'elenco dei clienti che si sono prenotati per il viaggio, e possono essere rappresentati attraverso una lista di cognomi;
- responsabile viaggio, identifica un dipendente della compagnia di viaggi, referente del viaggio.

Vacanza Invernale

- skipass, identifica il prezzo di uno skipass giornaliero;
- impianti sciistici, identifica l'elenco degli impianti sciistici convenzionati con l'agenzia di viaggio, rappresentabile come una lista;

Vacanza Estiva

- distanza, identifica la distanza, in metri, dal Resort alla spiaggia;
- escursioni marine, identifica l'elenco delle attività marittime a cui i clienti possono partecipare gratuitamente (snorkeling, parapendio, lezioni di surf, etc.).

La PT vorrebbe, inoltre, che da un viaggio fosse possibile determinare alcune utili informazioni.

Metodi Viaggio:

- **stampa**, funzione per stampare su schermo tutte le informazioni associate ad un determinato viaggio;
- **periodo**, funzione per determinare quanti giorni o mesi di vacanza sono stati programmati, come differenza della data di partenza e quella di ritorno;
- **guadagno**, funzione per determinare quale sia il ricavo netto dal viaggio, considerato che il 47% del prezzo di una prenotazione viene usato per coprire le spese del viaggio.

Si adattino le precedenti funzioni per i casi distinti di vacanza invernale e vacanze estiva. In particolare:

- il prezzo dello skipass pesa sulle spese del singolo ospite un ulteriore 15%;

- il prezzo delle escursioni marine pesa sulle spese del singolo ospite un ulteriore 10%.

Si realizzino le classi e sotto-classi necessarie per soddisfare le richieste della compagnia di viaggio. Inoltre, realizzare una funzione esterna che, dato il cognome di un preciso cliente ed una lista di tutti i viaggi organizzati dalla PT, determina i viaggi a cui il cliente ha partecipato e l'ammontare di denaro speso dal cliente per questi viaggi (la funzione non ritorna nulla).

Esercizio 10.8 (Ornitorinchi). Vogliamo fare esperimenti sugli ornitorinchi, detti anche platipi. Durante la stagione riproduttiva, ogni platipo depone un certo numero di uova. Vogliamo costruire un modello per tenere traccia di quante uova ha deposto un platipo in ogni stagione.

- implementare un classe che prende in input il nome del platipo e una lista contenente il numero di uova deposte in ogni stagione. Per esempio

```
1 Perry = Platipo('Perry', [3, 2, 4, 1, 2])
```

- Espandere la classe Platipo con i seguenti tre metodi:
 - `Uova_totali`, che ritorna il numero totali di uova deposte;
 - `Stagioni_feconde`, che ritorna il numero di stagioni riproduttive in cui il platipo ha deposto almeno un uovo;
 - `Deponi_uova` che aggiunge le uova deposte alla lista di quelle già deposte.

Sappiamo che i platipi possono deporre fino a 3 uova ogni stagione. Partendo dall'assunzione che ogni numero di uova ha la stessa probabilità di essere deposta (incluso zero), vogliamo modellare la dinamica riproduttiva di un gruppo di 3 platipi.

Creare 3 oggetti platipi, scegliendo un nome e un numero di uova deposte in una stagione (i.e. una lista di un elemento).

Simulare 10 stagioni riproduttive, per ogni stagione generare per ogni platipo un numero randomico tra 0 e 3 di uova deposte, ed aggiungerlo alla lista del platipo.

Una volta che la simulazione è terminata stampare il nome, uova totali e stagione feconde per ogni platipo con la seguente sintassi: "Billy ha depositato un totale di 10 uova, in 5 stagioni riproduttive feconde"

Nota bene. I numeri randomici possono essere generati con la funzione `randint(a,b)` presente nella libreria `random`; `a` e `b` sono l'estremo inferiore e superiore.

11 List comprehension

Esercizio 11.1. Si crei una lista di elementi nell'intervallo da 1000 a 2500 con step di 130, utilizzando la list comprehension. Si ragioni se la comprehension serva davvero in questo caso, o meno.

Esercizio 11.2. Si costruisca una funzione che prende in input una lista ed una costante c , e restituisca la lista degli stessi elementi aumentati di c , mediante comprehension.

Esercizio 11.3. Si costruisca una funzione che prende in input una lista ed una costante c , e restituisca la lista degli stessi elementi aumentati di c , mediante comprehension.

Esercizio 11.4. Utilizzando la comprehension, si costruisca una lista dai quadrati di una altra lista solamente se il valore del quadrato supera 100.

Esercizio 11.5. Si testi il seguente codice

```
1 [x for x in "This is my string"]
```

Utilizzando la comprehension e considerando le seguenti variabili

```
1 nums = [i for i in range(1,1001)]  
2  
3 string = "Stringa per l'esercizio pratico di list comprehension."
```

si:

- costruisca la lista dei numeri $x \in [1, 1000]$ divisibili per 8;
- costruisca la lista dei numeri $x \in [1, 1000]$ che contengono la cifra "6";
- contino il numero di spazi nella stringa `string`;
- rimuovano le vocali dalla stringa `string`;
- trovino tutte e sole le parole con meno di 5 lettere nella stringa `string` (per “tokenizzare” una frase nelle parole che la compongano si utilizzi `string.split(" ")`);
- trovino tutti i numeri $x \in [1, 1000]$ che sono divisibili per numeri ad una singola cifra escluso 1 (quindi per 2, ..., 9; si utilizzi una comprehension annidata in questo caso).

12 Iteratori

Esercizio 12.1 (Flusso pari). Si scriva una classe per un iteratore che genera il flusso

$$x, x + \delta, x + 2\delta, x + 3\delta, \dots, y$$

con x , δ ed $y > x$ input della classe.

Esercizio 12.2 (Flusso primi da lista). Si scriva una classe per un iteratore che restituisce solamente i numeri primi all'interno di una lista di interi (i.e., la classe memorizza la lista, e l'iteratore ci naviga sopra di conseguenza).

Esercizio 12.3 (Flusso da complemento). Si scriva una classe per un iteratore che restituisce solamente i numeri presenti all'interno di una lista di interi x , che non sono presenti all'interno di una altra lista di interi y (i.e., la classe memorizza due liste, e l'iteratore ci naviga sopra di conseguenza).

Esercizio 12.4 (Flusso doppio). Si scriva una classe per un iteratore che genera gli elementi della successione

$$a_i = \frac{2^i}{2i - 1}$$

Esercizio 12.5 (Discretizzazione funzione). Si consideri una funzione $f(x)$ discretizzata sull'intervallo $[a, b]$ sui punti

$$x_1, x_2, \dots, x_n$$

con $x_i = a + (i - 1)\delta_x$. Si costruisca un iteratore che generi i valori

$$f(x_1), f(x_2), \dots, f(x_n).$$

Esercizio 12.6 (Calendario itinerante). Realizzare una classe `Date` che presenta come attributi una generica data (giorno, mese, anno).

Inoltre, la classe presenta i seguenti metodi:

- `__init__`, costruttore per creare un oggetto della classe;
- `__str__`, funzione per stampare la data;
- `__iter__`, funzione per creare un iteratore, utilizzato per “muoversi” tra i giorni secondo un incremento di un giorno;
- `__next__`, funzione per realizzare la logica dell'iteratore. In particolare, l'iteratore deve essere in grado di riconoscere se l'anno di riferimento 'è un anno bisestile ed in caso modificare i giorni dei mesi opportuni. L'iteratore deve essere in grado di cambiare mese e/o anno dove 'è necessario (es. da 30/10/2020 si passa a 1/11/2020, oppure da 31/11/2020 si passa a 1/1/2021).

Utilizzare una funzione esterna che determina se l'anno di riferimento sia bisestile oppure no; si osservi che un anno viene considerato bisestile nei due casi seguenti:

- l'anno è divisibile per 400;
- l'anno è divisibile per 4, ma non per 100.

Il corpo principale del programma dovrà testare il corretto funzionamento della classe `Date` su tutte le casistiche che potrebbero essere di interesse.